VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 1: SystemVerilog Fundamentals and Data Types

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

How to Use These Notes 3

Part 1: Introduction to SystemVerilog 5
- 1.1 Introduction to SystemVerilog 5
- 1.2 Difference Between Verilog and SystemVerilog 8
- 1.3 Why SystemVerilog Was Introduced 11
- 1.4 SystemVerilog Applications in Design and Verification 12

Part 2: States, Nets, Variables and Built-in Types 19
- 2.1 Two-State and Four-State Data Types 19
- 2.2 Difference Between bit and logic 22
- 2.3 Difference Between reg, wire and logic 24
- 2.4 Net Types and Variable Types 26
- 2.5 Integer Data Types 29
- 2.6 Signed and Unsigned Data Types 34
- 2.7 Real and Realtime Data Types 38

Part 3: Packed, Unpacked and User-Defined Types 43
- 3.1 Packed and Unpacked Data Types 43
- 3.2 Packed Arrays and Unpacked Arrays 45
- 3.3 Structures (struct) 48
- 3.4 Unions (union) 53
- 3.5 Enumerated Data Types (enum) 57
- 3.6 typedef 62

Part 4: Casting, Default Values, Four-State Logic and Operators 66
- 4.1 Type Casting: Static and Dynamic 66
- 4.2 Default Values of Different Data Types 71
- 4.3 Four-State Logic Values: 0, 1, X, Z 74
- 4.4 Logical, Bitwise, Reduction and Shift Operators 79
- 4.5 Equality Operators: ==, ===, !=, !== 86
- 4.6 Wildcard Equality: ==? and !=? 89

How to Use These Notes

These notes are Module 1 of an eleven-module course that prepares you for SystemVerilog interviews for Design Verification (DV) roles. The goal is understanding, not memorising syntax: for each concept you should be able to say what it is, why it exists, how it behaves in a simulator, where it is used in a real testbench, and what mistakes people make with it.

How each topic is organised

1. The idea in plain English.
2. The technical definition, using the terms an interviewer expects.
3. Why it is needed.
4. How it works internally (memory layout, simulator behaviour).
5. Syntax.
6. Complete, runnable SystemVerilog code with the expected output.
7. A line-by-line explanation of that code.
8. Comparison tables against related concepts.
9. Common mistakes and interview traps.
10. Interview questions with answers.

Conventions used in the code

Convention	Meaning
`_t` suffix	A user-defined type made with `typedef` (for example `hdr_t`)
`_e` suffix	An enumerated type (for example `alu_op_e`)
`_u` suffix	A union type
`tb_` prefix	A small testbench module whose only purpose is to demonstrate a concept
`alu`, `alu_pkg`	The 8-bit ALU that is the running example DUT for the whole course
`clk`, `rst_n`	Clock, and active-low reset

About the code examples. Every example that carries a file name (such as `tb_states.sv`) is also saved as a separate file in the `examples/module-01/` folder of the repository, so you can paste it into any simulator. All examples use standard IEEE 1800 SystemVerilog and are written to run on the commercial simulators (Synopsys VCS, Cadence Xcelium, Siemens Questa). The free website EDA Playground gives you access to these simulators in a browser, and that is the easiest way to run the code.

> **Note:** The "Expected output" blocks in this module were worked out by hand from the rules in the IEEE 1800 standard; they were not captured from a simulator run. Run each example yourself. Doing so is the best way to learn, and if a simulator prints something different, working out why is exactly the kind of understanding an interviewer is testing. Where behaviour is known to differ between simulators (for example the exact layout printed by `%p`), the text says so.

Terms you will meet immediately

Term	Meaning
RTL	Register Transfer Level: the synthesizable description of the hardware
DUT	Design Under Test: the RTL block being verified
Testbench (TB)	Non-synthesizable code that drives the DUT and checks its outputs
Simulator	The software tool that executes the RTL and testbench (VCS, Xcelium, Questa)
Synthesis	Conversion of RTL into a gate-level netlist by a synthesis tool
LRM	Language Reference Manual: the IEEE 1800 standard document
Synthesizable	Can be converted into real hardware by a synthesis tool
Simulation-only	Understood by the simulator only; has no hardware equivalent

Part 1: Introduction to SystemVerilog

1.1 Introduction to SystemVerilog

In plain English

A chip is first described as text. Engineers write code that describes what the hardware should do, a tool turns that code into logic gates, and a factory turns the gates into silicon. A mistake found after the chip is manufactured costs millions of dollars and months of delay, so the code is tested very heavily before manufacturing. That testing is called **verification**, and it takes more engineering effort than writing the design itself.

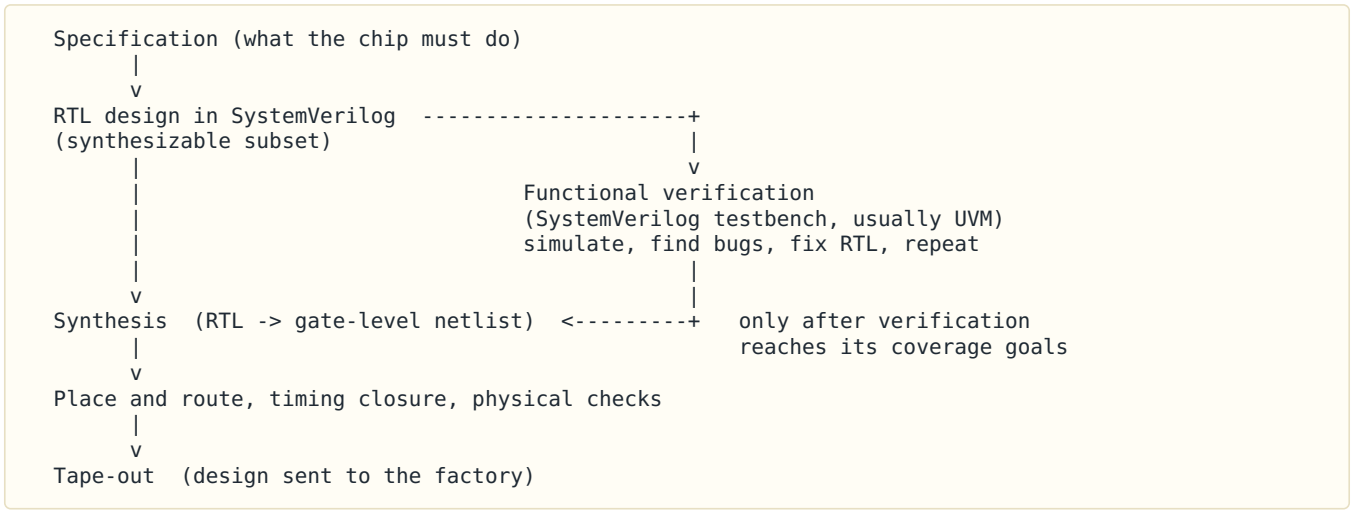
SystemVerilog is one language that serves both jobs. One part of the language describes hardware. A larger part of the language exists only to test that hardware: it can generate random stimulus, model expected behaviour, check results automatically and measure how much has been tested.

Technical definition

SystemVerilog is a **Hardware Description and Verification Language (HDVL)**, standardised as **IEEE 1800**. It is a superset of Verilog (IEEE 1364): every legal Verilog-2005 construct is legal SystemVerilog, and SystemVerilog adds

- richer data types (`logic`, `bit`, `int`, `enum`, `struct`, `union`, `typedef`, dynamic arrays, queues, associative arrays, strings),
- design constructs that state the designer's intent (`always_ff`, `always_comb`, `always_latch`, `unique`/`priority`, interfaces, packages),
- object-oriented programming (classes, inheritance, polymorphism),
- constrained-random stimulus generation (`rand`, `randc`, `constraint`, `randomize()`),
- functional coverage (`covergroup`, `coverpoint`, `cross`),
- assertions (immediate assertions and concurrent SystemVerilog Assertions, SVA),
- inter-process communication (mailboxes, semaphores, events, `fork`…`join`),
- a direct interface to C code (DPI).

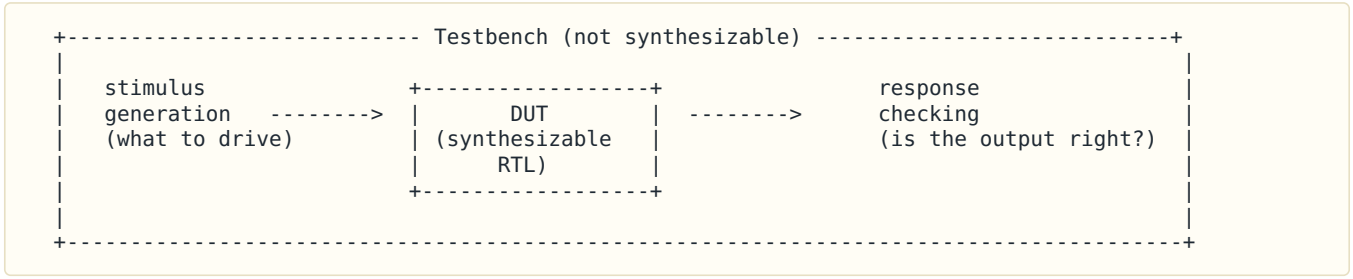
Where SystemVerilog sits in the chip design flow



A Design Verification engineer works in the box on the right. The job is to prove, before tape-out, that the RTL does what the specification says.

DUT and testbench

Everything in verification is built around two things:



- The **DUT** must be synthesizable, because it becomes hardware.
- The **testbench** never becomes hardware. It only runs in a simulator, so it may use the whole language.

This split explains most of the "why" questions about SystemVerilog data types. Hardware signals need values such as "unknown" and "not driven", so the DUT uses four-state types. Testbench bookkeeping (loop counters, transaction counts, scoreboards) does not, so the testbench mostly uses faster two-state types. You will see this theme throughout this module.

Your first SystemVerilog program

`tb_hello.sv`

```
module tb_hello;

  int unsigned count;
  string       name = "SystemVerilog";

  initial begin
    count = 3;
    repeat (count) begin
      $display("[%0t] Hello from %s", $time, name);
      #10;
    end
    $display("[%0t] Done", $time);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[0] Hello from SystemVerilog
[10] Hello from SystemVerilog
[20] Hello from SystemVerilog
[30] Done
```

Line by line

Line	Explanation
`module tb_hello;`	A module is the basic building block. A top-level testbench module has no ports because nothing is above it.
`int unsigned count;`	A 32-bit two-state unsigned variable. `int` does not exist in Verilog; it is a SystemVerilog type.
`string name = "SystemVerilog";`	`string` is a SystemVerilog type whose length changes automatically. In Verilog you had to use a fixed-width `reg` vector.
`initial begin`	An `initial` block starts at simulation time 0 and runs once. It is simulation-only.
`repeat (count)`	Executes the body `count` times.
`$display(...)`	A system task that prints a line. `%0t` prints the time with no padding, `%s` prints a string.
`$time`	A system function that returns the current simulation time.
`#10;`	Suspends this process for 10 time units. Simulation time advances; no real time is implied.
`$finish;`	Ends the simulation.

What a simulator does with your code

A simulator processes the source in three steps. Interviewers like to ask at which step a particular error is detected.

Step	What happens	Typical errors found here
Compilation	Each file is parsed and checked for syntax and type rules	Missing semicolon, undeclared identifier, assigning to a net in a procedural block
Elaboration	The design hierarchy is built: modules are instantiated, parameters resolved, ports connected	Port width mismatch, missing module, illegal parameter value
Simulation (run time)	Processes execute and simulation time advances	Null handle access, `$cast` failure, randomization failure, array index out of range, assertion failure

> **Key point:** SystemVerilog is "statically typed". The type of every variable is fixed at compile time. Only a few things are decided at run time: the size of dynamic arrays, queues and associative arrays, the object a class handle points to, and whether a `$cast` succeeds.

Interview questions

Q1. What is SystemVerilog? SystemVerilog is a hardware description and verification language standardised as IEEE 1800. It is a superset of Verilog. It adds design features such as `logic`, `always_ff`, `always_comb`, interfaces, enums and structs, and verification features such as classes, constrained randomization, functional coverage, assertions and inter-process communication.

Q2. Is SystemVerilog a design language or a verification language? Both. A synthesizable subset is used for RTL design. The complete language, including the non-synthesizable part (classes, randomization, coverage, mailboxes and so on), is used for verification.

Q3. What does "HDVL" mean? Hardware Description and Verification Language. Verilog and VHDL are HDLs. SystemVerilog adds the "V" because verification features are part of the same language instead of being a separate language.

1.2 Difference Between Verilog and SystemVerilog

A short history

Knowing the history lets you answer "why was it introduced?" with facts.

Year	Event
1984-85	Verilog created at Gateway Design Automation
1995	Verilog becomes IEEE 1364-1995
2001	IEEE 1364-2001 (Verilog-2001): `generate`, ANSI-style ports, signed arithmetic, `always @*`
2002-04	Accellera publishes SystemVerilog 3.0, 3.1 and 3.1a. Much of it comes from donated technology: the Superlog language (design extensions), OpenVera (testbench features), and assertion languages
2005	SystemVerilog becomes IEEE 1800-2005. Verilog is updated as IEEE 1364-2005
2009	IEEE 1800-2009 merges Verilog and SystemVerilog into one standard. From here on, "Verilog" is formally just a subset of SystemVerilog
2012, 2017, 2023	Later revisions of IEEE 1800. Most tools and interview questions assume 1800-2012 or 1800-2017

Comparison table

Aspect	Verilog (IEEE 1364)	SystemVerilog (IEEE 1800)
Category	HDL	HDVL (design and verification)
Main data types	`wire`, `reg`, `integer`, `real`, `time`	All Verilog types plus `logic`, `bit`, `byte`, `shortint`, `int`, `longint`, `string`, `enum`, `struct`, `union`, `class`
Number of states	Four-state only (0, 1, X, Z)	Four-state and two-state types
User-defined types	None	`typedef`, `enum`, `struct`, `union`, `class`
Arrays	Fixed-size only	Fixed-size, dynamic, associative, queues, with built-in methods
Strings	Stored in fixed-width `reg` vectors	Built-in dynamic `string` type with methods
Procedural blocks	`always`, `initial`	Adds `always_ff`, `always_comb`, `always_latch`, `final`
Programming style	Procedural, module based	Procedural plus object-oriented
Randomization	`$random` only, no constraints	`rand`, `randc`, constraint blocks, `randomize()`, `$urandom`, `$urandom_range`
Coverage	Code coverage from the tool only	Functional coverage built in: `covergroup`, `coverpoint`, `cross`
Assertions	None (use `if` plus `$display`)	Immediate and concurrent assertions (SVA)
Connecting blocks	Individual ports and wires	`interface`, `modport`, `clocking` block, `.*` and `.name` connections
Process communication	Named events only	Events, mailboxes, semaphores, `fork`…`join_any`/`join_none`, process control
Code sharing	`` `include `` and `` `define ``	Packages with `import`
Tasks and functions	Static by default, no `ref`, no default arguments, a function must return a value	`automatic` lifetime, `ref` arguments, default arguments, `void` functions, `return`
Loops and control	`for`, `while`, `repeat`, `forever`	Adds `foreach`, `do`…`while`, `break`, `continue`, `return`
Operators	No `++`, `+=`	`++`, `--`, `+=` and similar, `inside`, `==?`, `!=?`, streaming operators
C integration	PLI/VPI (complex)	DPI (simple function calls to and from C)
Typical use	RTL design, simple testbenches	RTL design and complete verification environments, base language of UVM

The same design in both languages

A 4-bit counter in Verilog:

```
module counter_v (clk, rst_n, en, count);
  input        clk, rst_n, en;
  output [3:0] count;
  reg    [3:0] count;                // must be reg: assigned in always

  always @(posedge clk or negedge rst_n) begin
    if (!rst_n)  count <= 4'b0000;
    else if (en) count <= count + 1;
  end
endmodule
```

The same counter in SystemVerilog:

```
module counter_sv (
  input  logic       clk,
  input  logic       rst_n,
  input  logic       en,
  output logic [3:0] count          // logic: no reg/wire decision needed
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)  count <= '0;       // '0 fills any width with zeros
    else if (en) count <= count + 1'b1;
  end
endmodule
```

What changed, and why each change matters:

Change	Benefit
`reg` replaced by `logic`	You no longer choose between `reg` and `wire` based on how the signal is assigned
`always` replaced by `always_ff`	States the intent "this is a flip-flop". Tools report an error if the block does not describe sequential logic, or if the same variable is also written from another process
`4'b0000` replaced by `'0`	Works for any width, so the code survives a width change

Interview questions

Q1. Give five differences between Verilog and SystemVerilog. Pick the ones you can explain: (1) SystemVerilog has two-state types such as `bit` and `int`; Verilog has only four-state types. (2) SystemVerilog has classes and OOP. (3) SystemVerilog has constrained randomization. (4) SystemVerilog has functional coverage and assertions built in. (5) SystemVerilog has interfaces to bundle signals. (6) SystemVerilog has dynamic arrays, queues and associative arrays. (7) SystemVerilog has `always_ff`, `always_comb` and `always_latch`.

Q2. Is Verilog code valid SystemVerilog? Yes, with one practical exception: SystemVerilog added many reserved keywords (`bit`, `logic`, `class`, `rand`, `interface`, `priority`, `do`, `final` and others). Old Verilog code that used one of these as a signal name will not compile as SystemVerilog until it is renamed, or the file is compiled with the `` `begin_keywords "1364-2005" `` directive.

Q3. Is Verilog still a separate standard? No. Since IEEE 1800-2009 there is one merged standard. The IEEE 1364 standard is no longer maintained.

1.3 Why SystemVerilog Was Introduced

The problem: the verification gap

Chip complexity grows much faster than the ability to verify it by hand. With Verilog, a testbench was a set of **directed tests**: the engineer wrote each stimulus value and each expected value manually. That works for a counter. It does not work for a processor or a network switch, where the number of possible scenarios is far too large to list. Projects were spending about 60 to 70 percent of their effort on verification and bugs were still reaching silicon.

Specific limitations of Verilog

Limitation in Verilog	Consequence	What SystemVerilog added
Only `$random`, with no way to constrain values	Random stimulus was mostly illegal or uninteresting	`rand` / `randc` variables, constraints, a constraint solver
No data structures	Packets and transactions were handled as many separate variables	`struct`, `class`, queues, dynamic and associative arrays
No object-oriented programming	Testbench code could not be reused or extended cleanly	Classes, inheritance, polymorphism
No way to measure what was tested	"Are we done?" could not be answered objectively	Functional coverage
No assertion language	Protocol checks were hand-written `always` blocks	SystemVerilog Assertions
Only four-state types	Slow, memory-hungry testbenches	Two-state types
`reg` versus `wire` confusion	Constant source of compile errors for beginners	`logic`
Ports connected signal by signal	Hundreds of repeated lines, easy to mis-connect	`interface`, `.*` connections
`always` does not state intent	Unintended latches, simulation and synthesis mismatches	`always_ff`, `always_comb`, `always_latch`
DUT and testbench race at clock edges	Testbench results depended on simulator ordering	`program` blocks and `clocking` blocks with defined sampling and driving regions
No good way to synchronise parallel threads	Hand-built handshakes with events	Mailboxes, semaphores, `fork` ... `join_any` / `join_none`

The second reason: too many languages

Before SystemVerilog, a typical project used Verilog or VHDL for RTL, a separate Hardware Verification Language such as Vera or *e* for the testbench, another language such as PSL for assertions, and C for reference models. Each needed its own tool, licence and training, and the boundaries between them caused problems. SystemVerilog unified design, testbench and assertions in one language, with DPI as a simple bridge to C.

> **How to say it in an interview:** "Verilog was adequate for describing hardware, but not for verifying large designs. It had no constrained randomization, no OOP, no functional coverage and no assertions, so teams used separate verification languages. SystemVerilog was introduced to unify design and verification in one language, and to raise verification productivity through constrained-random, coverage-driven testing."

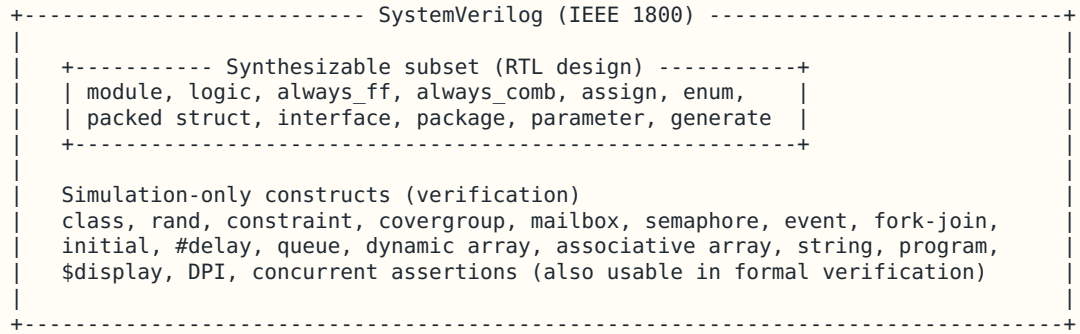
Interview questions

Q1. Why was SystemVerilog introduced when Verilog already existed? Use the answer in the box above, then give two concrete examples from the table.

Q2. What is constrained-random verification and why is it better than directed testing? In directed testing the engineer writes every stimulus by hand, so only the scenarios the engineer thought of are tested. In constrained-random verification the testbench generates random stimulus that is restricted to legal values by constraints. Thousands of scenarios are generated automatically, including corner cases nobody thought of. Functional coverage then measures which scenarios actually occurred. Directed tests are still used for the few specific cases that random stimulus does not reach. This is covered in depth in Modules 5, 6 and 10.

1.4 SystemVerilog Applications in Design and Verification

Two subsets of one language



What is synthesizable and what is not

Construct	Synthesizable?	Notes
`module`, ports, `parameter`, `localparam`, `generate`	Yes	
`logic`, `bit`, `wire`, `reg`, packed arrays	Yes	`bit` is accepted but hides X in simulation, so `logic` is preferred in RTL
`int`, `byte`, `integer`	Yes	Mostly used for loop variables and constants in RTL
`real`, `shortreal`, `realtime`, `time`	No	
`string`	No	Accepted only in parameters by some tools
`enum`, `typedef`, packed `struct`, packed `union`	Yes	Heavily used in modern RTL
Unpacked `struct`, fixed-size unpacked arrays	Yes	Supported by most tools

Construct	Synthesizable?	Notes
Dynamic arrays, queues, associative arrays	No	Size is not known at compile time
`always_ff`, `always_comb`, `always_latch`, `assign`	Yes	
`initial`	No	FPGA tools accept it for power-up values; ASIC flows ignore it
`#delay`, `wait`, `fork` … `join`	No	
`function`	Yes	If it contains no timing and no simulation-only constructs
`task`	Limited	Only if it contains no timing control
`interface`, `modport`	Yes	
`package`	Yes	When it contains synthesizable items (types, parameters, functions)
`class`, `rand`, `constraint`, `covergroup`	No	
`mailbox`, `semaphore`, `event`	No	
`program`, `clocking` block	No	Testbench constructs
Immediate and concurrent assertions	Ignored by synthesis	Used in simulation and formal verification
`$display`, `$finish`, `$random` and other system tasks	Ignored by synthesis	

> **Key point:** "Not synthesizable" does not mean "bad". The whole testbench is not synthesizable, and it does not need to be.

The course DUT: an 8-bit ALU

The following design is used as the running example throughout this course. In this module it shows the design-side features of SystemVerilog. From Module 6 onward we build a complete class-based testbench around it.

Specification

- Inputs `a` and `b` are 8 bits wide. Input `op` selects the operation.
- When `valid_in` is 1 at a rising clock edge, the ALU computes the result and presents it on the next cycle with `valid_out` = 1.
- `result` is 9 bits wide so that the carry of an addition (or the borrow of a subtraction) is visible in bit 8.
- `rst_n` is an asynchronous, active-low reset.

`op`	Name	`result`
000	`ALU_ADD`	`a + b` (bit 8 is the carry)
001	`ALU_SUB`	`a - b` (bit 8 is the borrow)
010	`ALU_AND`	`a & b`
011	`ALU_OR`	`a | b`
100	`ALU_XOR`	`a ^ b`

op	Name	result
101	ALU_NOT	~a
110	ALU_SHL	a shifted left by one (bit 8 receives the old a[7])
111	ALU_SHR	a shifted right by one

alu_pkg.sv

```
package alu_pkg;

  typedef enum logic [2:0] {
    ALU_ADD = 3'b000,
    ALU_SUB = 3'b001,
    ALU_AND = 3'b010,
    ALU_OR  = 3'b011,
    ALU_XOR = 3'b100,
    ALU_NOT = 3'b101,
    ALU_SHL = 3'b110,
    ALU_SHR = 3'b111
  } alu_op_e;

endpackage
```

alu.sv

```
module alu
  import alu_pkg::*;
(
  input  logic       clk,
  input  logic       rst_n,
  input  logic       valid_in,
  input  logic [7:0] a,
  input  logic [7:0] b,
  input  alu_op_e    op,
  output logic       valid_out,
  output logic [8:0] result
);

  logic [8:0] result_nxt;

  always_comb begin
    unique case (op)
      ALU_ADD: result_nxt = a + b;
      ALU_SUB: result_nxt = a - b;
      ALU_AND: result_nxt = {1'b0, a & b};
      ALU_OR : result_nxt = {1'b0, a | b};
      ALU_XOR: result_nxt = {1'b0, a ^ b};
      ALU_NOT: result_nxt = {1'b0, ~a};
      ALU_SHL: result_nxt = {a, 1'b0};
      ALU_SHR: result_nxt = {2'b00, a[7:1]};
    endcase
  end

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      valid_out <= 1'b0;
      result    <= '0;
    end
    else begin
      valid_out <= valid_in;
      if (valid_in) result <= result_nxt;
    end
  end

endmodule
```

Explanation of the design

Code	Explanation
`package alu_pkg; ... endpackage`	A package is a named container for declarations that several files need. Here it holds the opcode type so that the DUT and the testbench share one definition. Packages are covered in Module 11.
`typedef enum logic [2:0] {...} alu_op_e;`	Creates a new type with eight named values stored in 3 bits. Section 3.5 explains enums.
`import alu_pkg::*;`	Makes the names in the package visible inside the module. Placed before the port list so the port `op` can use the type.
`input logic [7:0] a`	An 8-bit four-state input.
`input alu_op_e op`	The port type is the enum, so waveforms show `ALU_ADD` instead of `000`.
`logic [8:0] result_nxt;`	Internal signal that carries the combinational result to the flip-flops.
`always_comb`	Combinational logic. The sensitivity list is inferred automatically, and the tool warns if a latch would be created.
`unique case (op)`	States that exactly one branch matches for every value. The simulator reports a violation if none or several match, and synthesis may optimise using that promise.

Code	Explanation
`result_nxt = a + b;`	The left side is 9 bits, so `a` and `b` are extended to 9 bits before the addition and the carry is kept. Section 4.4 explains this "context-determined width" rule.
`{1'b0, a & b}`	Concatenation. A zero is placed in bit 8 because logical operations produce no carry.
`{a, 1'b0}`	`a` shifted left by one position, written as a concatenation so that `a[7]` lands in bit 8.
`always_ff @(posedge clk or negedge rst_n)`	Sequential logic with asynchronous active-low reset.
`result <= '0;`	Nonblocking assignment, which is always used for flip-flops. `'0` sets every bit to 0.
`if (valid_in) result <= result_nxt;`	The result register only updates when the input is valid.

A first, directed testbench for the ALU

This is the Verilog style of testbench: one hand-written stimulus and one hand-written check. Its purpose here is to show simulation-only constructs next to the design. Module 9 replaces it with a layered, class-based testbench.

tb_alu_directed.sv

```
module tb_alu_directed;
  import alu_pkg::*;

  logic       clk = 0;
  logic       rst_n;
  logic       valid_in;
  logic [7:0] a, b;
  alu_op_e    op;
  logic       valid_out;
  logic [8:0] result;

  alu dut (.*);

  always #5 clk = ~clk;

  initial begin
    rst_n    = 0;
    valid_in = 0;
    a        = '0;
    b        = '0;
    op       = ALU_ADD;
    repeat (2) @(posedge clk);
    rst_n = 1;

    @(negedge clk);
    a        = 8'd200;
    b        = 8'd100;
    op       = ALU_ADD;
    valid_in = 1;

    @(negedge clk);
    valid_in = 0;
    if (valid_out !== 1'b1 || result !== 9'd300)
      $error("[%0t] FAIL: %0d + %0d gave %0d (valid_out=%b)", $time, a, b, result, valid_out);
    else
      $display("[%0t] PASS: %0d + %0d = %0d", $time, a, b, result);

    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
[30] PASS: 200 + 100 = 300
```

Execution flow

Time	What happens
0	`clk` starts at 0. The `initial` block drives reset active and gives every input a known value.
5, 15	Rising clock edges. The `repeat (2) @(posedge clk)` statement waits for both.
15	Reset is released (`rst_n = 1`).
20	Falling edge. The testbench drives `a` =200, `b` =100, `op` = `ALU_ADD` , `valid_in` =1. Driving on the falling edge keeps the stimulus stable around the rising edge and avoids a race with the DUT.
25	Rising edge. The DUT samples `valid_in` =1, stores `result` =300 and sets `valid_out` =1.
30	Falling edge. The testbench removes `valid_in` and checks the outputs. 200 + 100 = 300, which needs the ninth bit. The check passes.

Points to notice

Code	Explanation
`alu dut (.*);`	Implicit port connection: each port is connected to the signal of the same name. It only compiles when names and types match.
`always #5 clk = ~clk;`	Clock with a period of 10 time units. `#5` is simulation-only.
`!==` in the check	The case-inequality operator. It also detects X and Z. If `!=` were used and `result` were X, the comparison would be unknown, the `if` would not execute, and the test would wrongly pass. Section 4.5 explains this trap in full.
`$error`	Prints an error message with severity "error". The simulator counts these, which regression scripts use to mark the test as failed.

Where SystemVerilog is used in verification

Verification task	SystemVerilog feature	Module
Model one unit of stimulus (a packet, a bus transfer)	Transaction class	4, 6
Create many legal, varied stimuli	`rand` , constraints, `randomize()`	5
Pass transactions between testbench components	Mailbox	7
Run components in parallel	`fork` ... `join`	7
Connect the class-based testbench to DUT pins	Interface, virtual interface, clocking block	8
Organise the testbench	Generator, driver, monitor, scoreboard, environment	9
Check protocol rules continuously	Assertions	10
Measure what has been tested	Functional coverage	10

Verification task	SystemVerilog feature	Module
Industry-standard methodology built from all of the above	UVM, a class library written in SystemVerilog	11

Interview questions

Q1. Which SystemVerilog constructs are synthesizable? Modules, `logic` / `bit` / `wire` and packed arrays, `always_ff`, `always_comb`, `always_latch`, `assign`, enums, typedefs, packed structs and unions, interfaces, packages with synthesizable content, functions without timing, `generate`. Classes, randomization, coverage, dynamic arrays, queues, associative arrays, mailboxes, semaphores, delays and `initial` blocks are simulation-only.

Q2. What is the advantage of `always_ff` and `always_comb` over `always`? They state the design intent, and tools check it. `always_comb` infers its own sensitivity list (including signals read inside called functions), executes once at time 0 so outputs are consistent from the start, and tools warn if it would infer a latch. `always_ff` must describe sequential logic. Both forbid the same variable from being written by any other process, which catches multiple-driver bugs at compile time.

Q3. What does `unique case` do? It asserts that exactly one case item matches for every value of the case expression. In simulation a violation report is issued if no item matches or if more than one matches. For synthesis it means the items can be treated as mutually exclusive (parallel) and complete. `priority case` only asserts that at least one item matches, and keeps the priority order.

Q4. Why did the testbench use `!==` instead of `!=`? Because `!=` returns X when either operand contains X or Z, and an `if` treats X as false. A DUT output stuck at X would then never be reported. `!==` compares X and Z literally and always returns 0 or 1.

Part 2: States, Nets, Variables and Built-in Types

2.1 Two-State and Four-State Data Types

In plain English

A real wire in a digital circuit is normally at a low voltage (0) or a high voltage (1). But while designing and simulating hardware, two more situations must be represented:

- "I do not know whether this is 0 or 1." For example a flip-flop that has not been reset yet, or two gates driving opposite values onto the same wire.
- "Nothing is driving this wire at all." For example a bus where every tri-state driver is switched off.

A **four-state** data type can hold all four situations in each bit. A **two-state** data type can hold only 0 and 1 in each bit.

Technical definition

Value	Name	Meaning
`0`	Logic zero	Low, false
`1`	Logic one	High, true
`X`	Unknown	The simulator cannot determine whether the value is 0 or 1
`Z`	High impedance	The bit is not driven by anything (floating, tri-stated)

- **Four-state types** store 0, 1, X or Z per bit: `logic`, `reg`, `integer`, `time`, and all net types such as `wire`.
- **Two-state types** store 0 or 1 per bit: `bit`, `byte`, `shortint`, `int`, `longint`.

X and Z exist only in simulation. Real silicon has no "X" voltage; X is the simulator's way of saying "I cannot tell". Z is real: it is an undriven wire.

Why both kinds are needed

Need	Type to use	Reason
Model hardware signals accurately	Four-state	Uninitialised flip-flops, bus contention and floating buses must be visible, otherwise real bugs are hidden
Fast, compact testbench code	Two-state	A loop counter or packet count can never be "floating", so storing X and Z for it wastes memory and time

How it works internally

One bit that can take four values needs two bits of storage. Simulators use a pair of bits, traditionally called `aval` and `bval` (this is also the encoding exposed to C code through DPI and VPI):

```
value    aval  bval
-----    ----  ----
  0        0     0
  1        1     0
  Z        0     1
  X        1     1

logic [7:0] sig = 8'b10xz_01xx;          bit [7:0] cnt = 8'b1011_0110;

aval : 1 0 1 0 0 1 1 1                   bits : 1 0 1 1 0 1 1 0
bval : 0 0 1 1 0 0 1 1                   (one storage bit per bit)
(two storage bits per bit)
```

Consequences:

- A four-state variable uses twice the memory of a two-state variable of the same width.
- Every operation on four-state values must also compute the X/Z part, so it is slower.
- When a four-state value is copied into a two-state variable, `bval` is simply dropped: **every X and every Z becomes 0**. No warning is given.

Where X comes from in a simulation

Source of X	Example
Uninitialised four-state variable	A flip-flop that has no reset, before its first clocked assignment
Multiple drivers in conflict	Two `assign` statements drive 0 and 1 onto the same `wire`
Reading an array out of range	`mem[300]` when `mem` has 256 entries (four-state element type)
Unknown select or condition	`sel ? a : b` when `sel` is X and `a` differs from `b`
Arithmetic with an unknown operand	`a + b` where any bit of `a` is X gives a result that is entirely X
Division by zero	`a / 0` and `a % 0` return X
Undriven input	An undriven `wire` is Z; when it passes through a gate it becomes X
Timing violation in gate-level simulation	A setup or hold violation makes the flip-flop model output X

Complete example

tb_states.sv

```
module tb_states;

  logic [3:0] l4;   // four-state
  bit   [3:0] b2;   // two-state

  initial begin
    $display("default : l4=%b  b2=%b", l4, b2);

    l4 = 4'b1x0z;
    b2 = l4;        // X and Z are silently converted to 0
    $display("assigned: l4=%b  b2=%b", l4, b2);

    if ($isunknown(l4)) $display("l4 contains X or Z");
    if (!$isunknown(b2)) $display("b2 can never contain X or Z");

    l4 = 4'b1010;
    l4 = l4 + 4'b000x;   // one unknown bit poisons the whole sum
    $display("sum     : l4=%b", l4);
  end

endmodule
```

EXPECTED OUTPUT

```
default : l4=xxxx  b2=0000
assigned: l4=1x0z  b2=1000
l4 contains X or Z
b2 can never contain X or Z
sum     : l4=xxxx
```

Line by line

Line	Explanation
`logic [3:0] l4;`	Four-state. It has never been assigned, so each bit starts as X.
`bit [3:0] b2;`	Two-state. It starts as 0.
`l4 = 4'b1x0z;`	A literal may contain `x` and `z` digits.
`b2 = l4;`	`1` → 1, `x` → 0, `0` → 0, `z` → 0. The result is `1000`. Information has been lost.
`$isunknown(l4)`	System function that returns 1 if any bit of its argument is X or Z.
`l4 + 4'b000x`	For arithmetic operators, if any bit of any operand is X or Z, the entire result is X.

Which to use in a real verification environment

Location	Recommended	Why
RTL signals	`logic`	X must be visible so that reset and initialisation bugs are caught
Signals inside an `interface`	`logic`	They connect directly to DUT pins and must show X and Z
Fields sampled from the DUT by the monitor	`logic`	If the DUT drives X, the monitor must capture X so that the scoreboard can flag it
`rand` fields of a transaction class	`bit` / `int`	The constraint solver only generates 0 and 1, and two-state is faster

Location	Recommended	Why
Loop variables, counters, array indices, flags	`int`, `bit`	They never need X or Z

> **Interview trap:** "Why not use two-state types everywhere? They are faster." If a testbench samples a DUT output into a `bit` variable, an X on that output is converted to 0. The comparison against the expected value may then pass even though the DUT is broken. Signals that touch the DUT must be four-state, or must be checked with `$isunknown()` before conversion.

> **Common mistake:** Believing X means "either 0 or 1, and the simulator tries both". It does not. X is a distinct third value with its own propagation rules, and those rules are sometimes optimistic (see Section 4.3).

Interview questions

Q1. What are two-state and four-state data types? Two-state types hold only 0 and 1 per bit (`bit`, `byte`, `shortint`, `int`, `longint`). Four-state types hold 0, 1, X and Z per bit (`logic`, `reg`, `integer`, `time`, and nets such as `wire`).

Q2. Why were two-state types added? For simulation performance and lower memory use in testbench code, where X and Z have no meaning. They also make it easier to exchange data with C code through DPI.

Q3. What happens when a four-state value is assigned to a two-state variable? X and Z bits are converted to 0, with no error or warning.

Q4. How do you check whether a signal has X or Z? `$isunknown(sig)` returns 1 if any bit is X or Z. An alternative idiom is `(^sig) === 1'bx`, because the reduction XOR of a vector containing X or Z is X.

Q5. Does X exist in real hardware? No. Every node in silicon settles to some voltage. X is a simulation concept that represents lack of knowledge. Z does exist: it is a node with no active driver.

2.2 Difference Between bit and logic

Definition

Both `bit` and `logic` are **data types** for a single bit, and both can be turned into vectors by adding a packed dimension such as `[7:0]`. Both are unsigned by default. The only difference is the number of states.

Property	`bit`	`logic`
States	2 (0, 1)	4 (0, 1, X, Z)
Default value	0	X
Storage per bit	1 bit	2 bits
Simulation speed	Faster	Slower
Signed by default	No	No
Shows uninitialised state	No, appears as 0	Yes, appears as X
Can model tri-state (Z)	No	Yes

Property	`bit`	`logic`
Synthesizable	Yes	Yes
Typical use	Testbench variables, `rand` fields	RTL signals, interface signals, sampled DUT values

Syntax

```
bit          flag;        // 1 bit, two-state
bit   [7:0]  data_b;      // 8-bit two-state vector
logic        ready;       // 1 bit, four-state
logic [31:0] addr;        // 32-bit four-state vector
bit signed [7:0] offset;  // signed two-state vector
```

Why the default value matters: a reset bug

The following counter has a bug: the designer forgot the reset. Look at how each type shows the problem.

tb_bit_vs_logic.sv

```
module tb_bit_vs_logic;

  logic       clk = 0;
  logic [3:0] count_logic;   // four-state counter, no reset
  bit   [3:0] count_bit;     // two-state counter, no reset

  always #5 clk = ~clk;

  always_ff @(posedge clk) count_logic <= count_logic + 1'b1;
  always_ff @(posedge clk) count_bit   <= count_bit   + 1'b1;

  initial begin
    repeat (3) @(negedge clk);
    $display("count_logic = %b   count_bit = %b", count_logic, count_bit);
    $finish;
  end

endmodule
```

EXPECTED OUTPUT

```
count_logic = xxxx   count_bit = 0011
```

`count_logic` starts as X. X + 1 is X, so it stays X forever, and anybody looking at the waveform sees the missing reset immediately. `count_bit` starts at 0 and counts happily, so the simulation looks correct. The real chip, however, powers up with a random value in those flip-flops. The two-state type has hidden a real bug. This is the reason RTL is written with `logic`.

Interview questions

Q1. What is the difference between `bit` and `logic`? `bit` is two-state with a default of 0. `logic` is four-state with a default of X. Everything else (width, unsigned by default, usable as a vector) is the same.

Q2. Which would you use in a testbench and which in RTL? Why? `logic` in RTL and for anything connected to the DUT, so that uninitialised or undriven signals appear as X or Z. `bit` (and `int`) for internal testbench data such as transaction fields, counters and indices, for speed and because random values are always two-state.

Q3. Is `bit` synthesizable? Yes. But a simulation with `bit` starts flip-flops at 0 while the synthesised hardware starts at an unknown value, so simulation may not match hardware. `logic` is therefore preferred for RTL.

2.3 Difference Between reg, wire and logic

The Verilog problem

In Verilog every signal had to be declared as either a `wire` or a `reg`, and the choice depended on how the signal was assigned:

- assigned by an `assign` statement or driven by a module output → `wire`
- assigned inside an `always` or `initial` block → `reg`

Two things were wrong with this. First, the name `reg` suggests "register", but a `reg` assigned in a combinational `always @*` block becomes plain gates, not a flip-flop. Second, changing how a signal was assigned forced you to change its declaration.

The SystemVerilog solution

SystemVerilog introduced `logic`. A `logic` variable can be

- assigned in procedural blocks (`always_ff`, `always_comb`, `initial`, tasks, functions), **or**
- driven by **one** continuous `assign` statement, **or**
- driven by **one** output port of a module instance.

So in almost all cases you declare everything as `logic` and stop thinking about it. The single exception is a signal with **more than one driver** (a tri-state bus, an `inout` port). That still needs a net type such as `wire`, because only nets have the mechanism to resolve several drivers into one value.

Comparison table

Property	`wire`	`reg`	`logic`
What it is	A net type	A variable data type	A variable data type (the same type as `reg`)
States	4	4	4
Default value	Z (when undriven)	X	X
Stores a value	No, it shows what its drivers drive	Yes	Yes
Procedural assignment (`always`, `initial`)	Not allowed	Allowed	Allowed
Continuous assignment (`assign`)	Allowed, any number	Not allowed in Verilog; one allowed in SystemVerilog	One allowed
Multiple drivers	Allowed and resolved	Not allowed	Not allowed

Property	`wire`	`reg`	`logic`
Tri-state bus, `inout` port	Yes	No	No
Infers a flip-flop automatically	No	No, depends on how it is coded	No, depends on how it is coded
Introduced in	Verilog	Verilog	SystemVerilog

> **Key point:** In SystemVerilog, `reg` and `logic` are exactly the same data type. `logic` is simply the better name. New code uses `logic`; `reg` remains for backward compatibility.

Complete example with legal and illegal uses

tb_reg_wire_logic.sv

```
module tb_reg_wire_logic;

  logic       a, b;
  wire        w_and;        // net
  logic       l_or;         // variable driven by one continuous assignment
  logic       l_xor;        // variable assigned procedurally
  logic [3:0] sum;

  assign w_and = a & b;                 // legal: continuous assignment to a net
  assign l_or  = a | b;                 // legal: ONE continuous assignment to a variable

  always_comb l_xor = a ^ b;            // legal: procedural assignment to a variable

  // assign l_or = a;                   // ILLEGAL: second continuous driver on a variable
  // always_comb w_and = a;             // ILLEGAL: procedural assignment to a net
  // assign l_xor = a;                  // ILLEGAL: variable driven both procedurally and
continuously

  initial begin
    a = 1; b = 0;
    #1 $display("a=%b b=%b : and=%b or=%b xor=%b", a, b, w_and, l_or, l_xor);
    a = 1; b = 1;
    #1 $display("a=%b b=%b : and=%b or=%b xor=%b", a, b, w_and, l_or, l_xor);
  end

endmodule
```

EXPECTED OUTPUT

```
a=1 b=0 : and=0 or=1 xor=1
a=1 b=1 : and=1 or=1 xor=0
```

The three commented lines are the three classic compile errors. Remove a comment marker and compile to see the message your simulator gives; recognising that message is useful when debugging.

Interview questions

Q1. What is the difference between `reg`, `wire` and `logic`? `wire` is a net: it has no storage, it reflects the resolved value of its drivers, it can have several drivers, and it cannot be assigned in a procedural block. `reg` is a

Verilog variable: it stores a value and is assigned procedurally. `logic` is the SystemVerilog replacement for `reg`: the same four-state variable type, which may additionally be driven by a single continuous assignment or a single port, so it can replace both `reg` and `wire` everywhere except where multiple drivers are needed.

Q2. Does `reg` always become a flip-flop? No. It becomes a flip-flop only if it is assigned in a clocked block. In a combinational block it becomes combinational logic, or a latch if it is not assigned on every path.

Q3. Can `logic` replace `wire` everywhere? No. A signal with more than one driver, such as a bidirectional or tri-state bus, must be a net. `inout` ports must be nets.

Q4. What happens if two `assign` statements drive the same `logic` signal? Compile (or elaboration) error, because a variable may have at most one continuous driver. With a `wire`, the same code compiles, and a conflict shows up as X at run time. This is an advantage of `logic`: accidental multiple drivers are caught at compile time.

2.4 Net Types and Variable Types

Two independent properties of every signal

This is the idea that makes the previous section precise. Every signal ("data object") has two separate properties:

1. Its **kind**: is it a **net** or a **variable**?
2. Its **data type**: what set of values can it hold (`logic`, `bit`, `int`, an enum, a struct...)?

```
        kind            data type               name
        ----            ---------               ----
        wire            logic [7:0]             bus;       // net of type logic (written in full)
        wire            [7:0]                   bus;       // same thing: data type defaults to logic
        var             logic [7:0]             data;      // variable of type logic (written in full)
                        logic [7:0]             data;      // same thing: kind defaults to var
        var             bit                     flag;      // variable of type bit
```

So `wire` is not a data type; it is a kind of net whose default data type is `logic`. And `logic` is not a kind; it is a data type, which by default creates a variable. This is the technically exact answer to "what is the difference between `wire` and `logic`?".

Nets

A net models a physical connection. It does not store anything (except `trireg`). Its value is continuously computed from all of its drivers by a **resolution function**.

Net type	Behaviour	Typical use
`wire`, `tri`	Identical to each other. Multiple drivers are resolved; a conflict gives X	General connections; `tri` documents a tri-state bus
`uwire`	"Unresolved wire": only one driver allowed, more is an error	Enforcing single-driver nets
`wand`, `triand`	Wired-AND: if any driver is 0 the result is 0	Open-drain / open-collector buses
`wor`, `trior`	Wired-OR: if any driver is 1 the result is 1	Wired-OR logic
`tri0`	Like `tri`, but reads 0 when undriven	Net with pull-down resistor
`tri1`	Like `tri`, but reads 1 when undriven	Net with pull-up resistor (I2C `sda`, `scl`)

Net type	Behaviour	Typical use
`trireg`	Keeps its last driven value when undriven; defaults to X	Capacitive storage nodes
`supply0`, `supply1`	Constant 0 or 1 at supply strength	Ground and power

Resolution of two drivers on a `wire` or `tri`

Driver A \ Driver B	0	1	X	Z
0	0	X	X	0
1	X	1	X	1
X	X	X	X	X
Z	0	1	X	Z

Read it like this: Z means "not driving", so the other driver wins. Two drivers that agree give that value. Two drivers that disagree give X, which in real hardware is a short circuit (bus contention).

Variables

A variable stores a value. It keeps that value until the next assignment. Variables include everything declared with a data type and no net keyword: `logic`, `reg`, `bit`, `byte`, `shortint`, `int`, `longint`, `integer`, `time`, `real`, `string`, enums, structs, arrays, class handles.

The assignment rules

Rule	Net	Variable
Procedural assignment (`always`, `initial`, task, function)	Never	Yes, from any number of statements
Continuous assignment (`assign`)	Yes, any number	Yes, but only one, and then no procedural assignment
Driven by a module instance output port	Yes, any number	Yes, but only one
Multiple drivers	Resolved	Error
Value when never assigned	Z (`trireg`: X)	Default of its data type (X for four-state, 0 for two-state)
Can be initialised in its declaration	`wire w = a & b;` is a continuous assignment	`logic v = 1'b0;` is a one-time initial value

> **Interview trap:** `wire w = a & b;` and `logic v = a & b;` look alike and behave completely differently. The first is a continuous assignment: `w` follows `a & b` for the whole simulation. The second is an initialisation: `v` gets the value of `a & b` once, at time 0 (probably X), and never updates.

Port kinds

Port declaration	What you get
`input a`	Net (`wire logic`)

Port declaration	What you get
`input logic a`	Still a net: for `input` and `inout` ports the kind defaults to `wire` even when a data type is written
`output a`	Net
`output logic a`	Variable: for `output` ports, writing a data type makes it a variable
`inout wire a`	Net. `inout` ports must be nets

In practice you write `input logic` and `output logic` everywhere and it works. The table matters only for the interview question "is `input logic a` a net or a variable?".

Implicit nets

If a name that was never declared appears in a port connection or on the left of an `assign`, Verilog silently creates a 1-bit `wire`. A typing mistake therefore creates a new unconnected 1-bit net instead of an error. Adding `` `default_nettype none `` at the top of a file turns this into a compile error, which is why many companies require it in RTL.

Complete example: a tri-state bus

tb_net_resolution.sv

```
module tb_net_resolution;

  logic       en_a, en_b;
  logic [3:0] data_a, data_b;
  wire  [3:0] bus;                          // must be a net: it has two drivers

  assign bus = en_a ? data_a : 4'bzzzz;     // driver A
  assign bus = en_b ? data_b : 4'bzzzz;     // driver B

  initial begin
    data_a = 4'b1100;
    data_b = 4'b1010;

    en_a = 0; en_b = 0; #1 $display("nobody drives : bus=%b", bus);
    en_a = 1; en_b = 0; #1 $display("A drives      : bus=%b", bus);
    en_a = 0; en_b = 1; #1 $display("B drives      : bus=%b", bus);
    en_a = 1; en_b = 1; #1 $display("A and B drive : bus=%b", bus);
  end

endmodule
```

EXPECTED OUTPUT

```
nobody drives : bus=zzzz
A drives      : bus=1100
B drives      : bus=1010
A and B drive : bus=1xx0
```

Explanation

- `bus` is declared as `wire` because it has two `assign` statements. Declaring it as `logic` would be a compile error.

- Each driver either drives its data or drives Z ("releases the bus").
- When both are disabled the bus floats: `zzzz`.
- When both drive, each bit is resolved separately using the table above. Bit 3 is 1 and 1 → 1. Bit 2 is 1 and 0 → X. Bit 1 is 0 and 1 → X. Bit 0 is 0 and 0 → 0. Result `1xx0`.

> **Verification application:** A bidirectional pin such as I2C `sda` is declared as a net inside the interface. The testbench driver cannot assign to a net from a class, so the interface contains a `logic` variable (for example `sda_drv`) that the class writes, plus `assign sda = sda_oe ? sda_drv : 1'bz;`. This pattern is asked about frequently and is shown in Module 8.

Interview questions

Q1. What is the difference between a net and a variable? A net represents a connection, has no storage, takes the resolved value of its drivers, may have multiple drivers, and cannot be assigned procedurally. A variable stores a value, is assigned procedurally (or by exactly one continuous assignment or port), and cannot have multiple drivers.

Q2. Is `logic` a net or a variable? Neither: `logic` is a data type. A declaration `logic x;` creates a variable of type `logic`. A declaration `wire logic x;` (or just `wire x;`) creates a net of type `logic`.

Q3. What is the default value of a `wire`? Of a `logic`? An undriven `wire` is Z. An unassigned `logic` variable is X.

Q4. What is the difference between `wire` and `tri`? None in behaviour. `tri` is used to document that the net is intended to have several tri-state drivers.

Q5. What are `wand` and `wor`? Nets with a wired-AND or wired-OR resolution function. On a `wand`, any driver at 0 forces the net to 0. On a `wor`, any driver at 1 forces the net to 1. They model open-drain style buses.

Q6. What does `` `default_nettype none `` do? It disables implicit net declaration, so that an undeclared (often misspelled) signal name is a compile error instead of a silently created 1-bit wire.

2.5 Integer Data Types

Overview

"Integer data types" (the standard calls them integral types) are all the types that hold whole numbers as a group of bits.

Type	Width	States	Signed by default	Default value	Range	From Verilog?
`bit`	1 bit, or user-defined vector	2	No	0	0 to 2^N - 1	No
`logic`	1 bit, or user-defined vector	4	No	X	0 to 2^N - 1	No
`reg`	1 bit, or user-defined vector	4	No	X	0 to 2^N - 1	Yes
`byte`	8	2	Yes	0	-128 to 127	No

Type	Width	States	Signed by default	Default value	Range	From Verilog?
`shortint`	16	2	Yes	0	-32,768 to 32,767	No
`int`	32	2	Yes	0	-2,147,483,648 to 2,147,483,647	No
`longint`	64	2	Yes	0	-2^63 to 2^63 - 1	No
`integer`	32	4	Yes	X	-2,147,483,648 to 2,147,483,647	Yes
`time`	64	4	No	X	0 to 2^64 - 1	Yes

A way to remember it: the four C-like types (`byte`, `shortint`, `int`, `longint`) are two-state and signed. The two old Verilog types (`integer`, `time`) are four-state. The single-bit types (`bit`, `logic`, `reg`) are unsigned and are the only ones whose width you choose.

Memory representation

```
  bit             [ b ]                                   1 storage bit
  logic           [ a | b ]                               2 storage bits (aval, bval)

  byte            [ S | 6 5 4 3 2 1 0 ]                   8 bits, S = sign bit (bit 7)
  shortint        [ S | 14 ............... 0 ]            16 bits
  int             [ S | 30 ............................. 0 ]            32 bits
  longint         [ S | 62 ............................................ 0 ]   64 bits

  integer         [ S | 30 ............................. 0 ]   aval   32 bits
                  [                                       ]   bval   32 bits  (64 storage bits)

  time            [ 63 ........................................ 0 ]   aval   64 bits
                  [                                           ]   bval   64 bits  (128 storage
bits)
```

Signed types use **two's complement**, described in Section 2.6.

Each type in detail

bit

- **Declaration:** `bit flag;` `bit [7:0] data;` `bit [3:0][7:0] word;`
- **States / default:** two-state, 0. Unsigned unless declared `bit signed`.
- **Use:** flags, `rand` transaction fields of any width (`rand bit [15:0] addr;`), masks. It is the most used type in class-based testbenches.

logic

- **Declaration:** `logic ready;` `logic [31:0] rdata;`
- **States / default:** four-state, X. Unsigned unless declared `logic signed`.
- **Use:** all RTL signals, all interface signals, any testbench field that stores a value sampled from the DUT.

reg

- Identical to `logic`. Found in Verilog code and older testbenches. Do not use it in new code.

byte

- **Declaration:** `byte b;` `byte unsigned ub;`
- **States / default:** two-state, 0, **signed**, 8 bits.
- **Use:** characters (a `string` is a sequence of bytes), C-style data, DPI arguments matching C `char`.
- **Warning:** because it is signed, `byte` holds -128 to 127. For a payload byte that should hold 0 to 255, use `bit [7:0]` or `byte unsigned`.

shortint

- **Declaration:** `shortint s;` `shortint unsigned us;`
- **States / default:** two-state, 0, signed, 16 bits.
- **Use:** rare. Mostly to match C `short` in DPI calls, or 16-bit signed samples in DSP testbenches.

int

- **Declaration:** `int i;` `int unsigned count;`
- **States / default:** two-state, 0, signed, 32 bits.
- **Use:** the default choice for loop variables, counters, array indices, sizes, IDs, error counts.

longint

- **Declaration:** `longint big;` `longint unsigned ubig;`
- **States / default:** two-state, 0, signed, 64 bits.
- **Use:** values that can exceed about 2 billion: byte counters in long simulations, 64-bit addresses, timestamps in small time units, DPI arguments matching C `long long`.

integer

- **Declaration:** `integer n;`
- **States / default:** four-state, X, signed, 32 bits.
- **Use:** legacy Verilog code. In new testbench code `int` is preferred because it is faster and starts at 0. `integer` is appropriate only when you need a 32-bit signed number that can also show X.

time

- **Declaration:** `time t_start;`
- **States / default:** four-state, X, unsigned, 64 bits.
- **Use:** storing simulation timestamps returned by `$time`, for example to measure latency. Covered with `realtime` in Section 2.7.

int versus integer

Property	`int`	`integer`
States	2	4
Default value	0	X
Width	32	32
Signed	Yes	Yes
Speed and memory	Better	Worse

Property	`int`	`integer`
Origin	SystemVerilog	Verilog

Changing signedness and building your own widths

```
int unsigned       count;      // 32-bit unsigned, 0 to 4,294,967,295
byte unsigned      ubyte;      // 8-bit unsigned, 0 to 255
bit signed [11:0]  sample;     // 12-bit signed two-state
logic signed [7:0] offset;     // 8-bit signed four-state
// byte [3:0] x;               // ILLEGAL: types with a fixed width cannot take a packed dimension
```

Complete example

tb_integer_types.sv

```
module tb_integer_types;

  bit      v_bit;
  logic    v_logic;
  byte     v_byte;
  shortint v_shortint;
  int      v_int;
  longint  v_longint;
  integer  v_integer;
  time     v_time;

  initial begin
    $display("--- default values ---");
    $display("bit=%b logic=%b byte=%0d shortint=%0d", v_bit, v_logic, v_byte, v_shortint);
    $display("int=%0d longint=%0d integer=%0d time=%0d", v_int, v_longint, v_integer, v_time);

    $display("--- width in bits ---");
    $display("bit=%0d logic=%0d byte=%0d shortint=%0d",
             $bits(v_bit), $bits(v_logic), $bits(v_byte), $bits(v_shortint));
    $display("int=%0d longint=%0d integer=%0d time=%0d",
             $bits(v_int), $bits(v_longint), $bits(v_integer), $bits(v_time));

    $display("--- overflow wraps around ---");
    v_byte = 127;          v_byte++;     $display("byte     127 + 1        = %0d", v_byte);
    v_shortint = 32767;    v_shortint++; $display("shortint 32767 + 1      = %0d", v_shortint);
    v_int = 2147483647;    v_int++;      $display("int      2147483647 + 1 = %0d", v_int);
    v_byte = 200;                        $display("byte     assigned 200   = %0d", v_byte);

    $display("--- four-state to two-state ---");
    v_integer = 'x;
    v_int     = v_integer;
    $display("integer=%0d copied into int=%0d", v_integer, v_int);
  end

endmodule
```

EXPECTED OUTPUT

```
--- default values ---
bit=0 logic=x byte=0 shortint=0
int=0 longint=0 integer=x time=x
--- width in bits ---
bit=1 logic=1 byte=8 shortint=16
int=32 longint=64 integer=32 time=64
--- overflow wraps around ---
byte     127 + 1        = -128
shortint 32767 + 1      = -32768
int      2147483647 + 1 = -2147483648
byte     assigned 200   = -56
--- four-state to two-state ---
integer=x copied into int=0
```

Explanation

Code	Explanation
Default values	The two-state types start at 0. `logic`, `integer` and `time` start at X. With `%d`, a value whose bits are all X prints as a lowercase `x`; a value with only some X bits prints as an uppercase `X`.
`$bits(v)`	System function returning the number of bits a variable or type occupies.
`v_byte = 127;` `v_byte++;`	127 is `0111_1111`. Adding 1 gives `1000_0000`, which in two's complement is -128. There is no overflow error in SystemVerilog; the value wraps silently.
`v_byte = 200;`	200 is `1100_1000`. In an 8-bit signed type the top bit is the sign, so this pattern means 200 - 256 = -56.
`v_integer = 'x;`	`'x` sets every bit to X. Copying it to `int` turns every X into 0.

> **Interview trap:** What does this loop do? `for (byte i = 0; i < 200; i++) ...` It never ends. A `byte` cannot exceed 127; after 127 it wraps to -128, which is still less than 200. The same bug appears with `bit [3:0] i; for (i = 0; i < 16; i++)`: a 4-bit variable is always less than 16. Use `int` for loop variables.

Which type to use: a quick guide

Situation	Type
RTL signal, interface signal	`logic [N-1:0]`
Random field of a transaction	`rand bit [N-1:0]`
Loop variable, counter, index, size	`int`
Count that may exceed 2^31	`longint`
Payload byte, 0 to 255	`bit [7:0]` (not `byte`)
Signed quantity of a specific width	`bit signed [N-1:0]` or `logic signed [N-1:0]`
Timestamp	`time` or `realtime`
Value passed to or from C through DPI	The matching C-like type (`byte`, `shortint`, `int`, `longint`)

Interview questions

Q1. List the integer data types with their sizes. `bit` and `logic` (1 bit, or a user-defined vector), `byte` 8, `shortint` 16, `int` 32, `longint` 64, `integer` 32, `time` 64.

Q2. Which of them are four-state? `logic`, `reg`, `integer`, `time`. The others are two-state.

Q3. Which are signed by default? `byte`, `shortint`, `int`, `longint`, `integer`. Unsigned by default: `bit`, `logic`, `reg`, `time`.

Q4. What is the difference between `int` and `integer`? Both are 32-bit signed. `int` is two-state with default 0. `integer` is four-state with default X.

Q5. What is the range of `byte`? What is stored if you assign 255 to it? -128 to 127. Assigning 255 (`1111_1111`) stores -1.

Q6. What is the difference between `byte` and `bit [7:0]`? Both are 8-bit two-state. `byte` is signed (-128 to 127); `bit [7:0]` is unsigned (0 to 255). They hold the same bit patterns but behave differently in comparisons, in arithmetic, when extended to a wider type, and when printed with `%d`.

Q7. Can you write `int [7:0] x;`? No. Types with a predefined width (`byte`, `shortint`, `int`, `longint`, `integer`, `time`) cannot take a packed dimension. You can make an unpacked array of them: `int x [8];`.

2.6 Signed and Unsigned Data Types

In plain English

The bits stored in a variable are just a pattern. "Signed" or "unsigned" is the rule for interpreting that pattern as a number. The 8-bit pattern `1111_1111` means 255 if the variable is unsigned, and -1 if it is signed. The stored bits are identical.

Two's complement

Signed integers use two's complement. The most significant bit (MSB) has a negative weight:

```
8-bit pattern :    b7     b6    b5    b4    b3    b2    b1    b0
unsigned weight:   128    64    32    16     8     4     2     1
signed weight  :  -128    64    32    16     8     4     2     1

1111_1111 unsigned = 128+64+32+16+8+4+2+1 = 255
1111_1111 signed   = -128+64+32+16+8+4+2+1 = -1
1000_0000 signed   = -128        (most negative value)
0111_1111 signed   = 127         (most positive value)

To negate a number: invert every bit, then add 1.
    5 = 0000_0101  ->  invert: 1111_1010  ->  add 1: 1111_1011 = -5
```

What signedness actually changes

Signedness affects exactly four things:

Operation	Unsigned	Signed
Extension to a wider size	Zero-extended (padded with 0)	Sign-extended (padded with copies of the MSB)
Relational comparison `<`, `>`, `<=`, `>=`	Compared as magnitudes	Compared as two's complement numbers
Arithmetic right shift `>>>`	Fills with 0	Fills with the sign bit
Printing with `%d`, and `*`, `/`, `%` results	Non-negative value	May be negative

Addition, subtraction, bitwise operations and `==` on equal-width operands produce the same bits either way.

Syntax

```
logic signed [7:0] s_vec;       // signed vector
bit   signed [15:0] s_bits;
int unsigned        u_int;      // remove the default sign
byte unsigned       u_byte;

8'sd5          // signed sized literal
4'sb1111       // signed 4-bit literal with value -1
-8'sd3         // signed literal, value -3
5              // unsized decimal literal: a signed 32-bit int
'd5            // based literal without s: UNSIGNED 32-bit
8'hFF          // sized based literal without s: unsigned

signed'(expr)     // cast: reinterpret as signed
unsigned'(expr)   // cast: reinterpret as unsigned
$signed(expr)     // Verilog-2001 equivalent
$unsigned(expr)
```

The rules of signed expressions

These five rules explain nearly every signed/unsigned interview question.

1. **An expression is signed only if every operand is signed.** If any one operand is unsigned, all operands are treated as unsigned.
2. **Extension depends on the right-hand side, never on the left-hand side.** When a narrow value is assigned to a wider variable, it is sign-extended if the *source expression* is signed and zero-extended if it is unsigned. The type of the destination is irrelevant.
3. **A bit-select or part-select is always unsigned**, even if it selects the whole vector: `s_vec[7:0]` is unsigned.
4. **A concatenation is always unsigned.**
5. **A based literal is unsigned unless it has the `s` flag.** `4'b1111` is 15; `4'sb1111` is -1. A plain decimal literal such as `15` is signed.

Complete example

tb_signed.sv

```
module tb_signed;

  byte             s8;     // signed by default
  bit        [7:0] u8;     // unsigned by default
  bit signed [7:0] sv8;    // vector made signed
  int              i;
  int unsigned     ui;
  longint          li;

  initial begin
    s8 = 8'hFF;  u8 = 8'hFF;  sv8 = 8'hFF;
    $display("1. same bits FF     : s8=%0d  u8=%0d  sv8=%0d", s8, u8, sv8);

    i = s8;  $display("2. int = byte       : %0d  (sign-extended)", i);
    i = u8;  $display("3. int = bit [7:0]  : %0d  (zero-extended)", i);

    i  = -1;
    ui = 1;
    if (i < ui) $display("4. -1 < 1           : true");
    else        $display("4. -1 < 1           : false, -1 was treated as %0d", unsigned'(i));

    s8 = -8;
    $display("5. -8 >>> 1 = %0d    -8 >> 1 = %0d", s8 >>> 1, s8 >> 1);
    u8 = 8'hF8;
    $display("6. unsigned F8 >>> 1 = %0d", u8 >>> 1);

    $display("7. s8 = %0d   s8[7:0] = %0d", s8, s8[7:0]);

    i = 4'sb1111;  $display("8. int = 4'sb1111   : %0d", i);
    i = 4'b1111;   $display("9. int = 4'b1111    : %0d", i);

    li = -1;              $display("10. longint = -1            : %0d", li);
    li = unsigned'(-1);   $display("11. longint = unsigned'(-1) : %0d", li);
  end

endmodule
```

EXPECTED OUTPUT

```
1. same bits FF     : s8=-1  u8=255  sv8=-1
2. int = byte       : -1  (sign-extended)
3. int = bit [7:0]  : 255  (zero-extended)
4. -1 < 1           : false, -1 was treated as 4294967295
5. -8 >>> 1 = -4    -8 >> 1 = 124
6. unsigned F8 >>> 1 = 124
7. s8 = -8   s8[7:0] = 248
8. int = 4'sb1111   : -1
9. int = 4'b1111    : 15
10. longint = -1            : -1
11. longint = unsigned'(-1) : 4294967295
```

Line by line

Line	Explanation
1	The same bit pattern `1111_1111` prints as -1 for the signed variables and 255 for the unsigned one.
2	The source `s8` is signed, so its MSB (1) is copied into the upper 24 bits: `FFFF_FFFF` = -1.
3	The source `u8` is unsigned, so the upper 24 bits are filled with 0: `0000_00FF` = 255. The destination is a signed `int` in both cases; only the source decides (rule 2).

Line	Explanation
4	`i` is signed and `ui` is unsigned. Rule 1: the comparison is unsigned. The pattern of -1 is `FFFF_FFFF`, which as an unsigned number is 4,294,967,295, and that is not less than 1.
5	-8 is `1111_1000`. `>>>` on a signed operand shifts in copies of the sign bit: `1111_1100` = -4. `>>` always shifts in 0: `0111_1100` = 124.
6	`>>>` on an unsigned operand behaves exactly like `>>`. The operator is "arithmetic" only when its left operand is signed.
7	A part-select is unsigned (rule 3), so the same eight bits print as 248.
8, 9	The `s` in the literal decides whether the 4-bit value `1111` is extended with ones (giving -1) or zeros (giving 15).
10	The literal `-1` is a signed 32-bit value; it is sign-extended to 64 bits and stays -1.
11	`unsigned'(-1)` is the 32-bit pattern `FFFF_FFFF` regarded as unsigned. It is zero-extended to 64 bits, giving 4,294,967,295.

Interview trap: "I declared the destination as signed, so the value will be sign-extended." Wrong. Extension is decided by the signedness of the expression on the right-hand side. `int i = u8;` zero-extends even though `i` is signed.

Common mistake: Mixing one unsigned operand into signed arithmetic. In `result = s_a * s_b + u_offset;` the single unsigned `u_offset` makes the entire expression unsigned, so `s_a` and `s_b` are zero-extended instead of sign-extended and the product is wrong for negative inputs. Fix: `result = s_a * s_b + signed'({1'b0, u_offset});` or cast as appropriate.

Verification application: A scoreboard's reference model must use the same signedness as the DUT. If the DUT has a signed multiplier and the reference model computes with unsigned `bit [7:0]` fields, every negative operand produces a false mismatch. When a comparison fails only for values with the MSB set, suspect signedness first.

Interview questions

Q1. How are negative numbers represented in SystemVerilog? In two's complement.

Q2. What is the difference between `>>` and `>>>`? `>>` is a logical right shift and always fills the vacated bits with 0. `>>>` is an arithmetic right shift: it fills with the sign bit when the left operand is signed, and with 0 when the left operand is unsigned.

Q3. `logic signed [3:0] a = -2; logic [7:0] b; b = a;` What is `b`? `a` is `1110` and is signed, so it is sign-extended: `b` = `1111_1110` = 254 as an unsigned value. The destination being unsigned does not prevent sign extension.

Q4. What is the result of comparing a signed and an unsigned operand? Both are treated as unsigned. A negative signed value becomes a large positive value, so `-1 < 1` is false when one side is unsigned.

Q5. What is the difference between `8'd200` and `8'sd200`? Both store the pattern `1100_1000`. `8'd200` is unsigned and has the value 200. `8'sd200` is signed and has the value -56.

Q6. How do you convert between signed and unsigned? With the casts `signed'(x)` and `unsigned'(x)`, or the system functions `$signed(x)` and `$unsigned(x)`. They do not change any bit; they change how the bits are interpreted in the surrounding expression.

2.7 Real and Realtime Data Types

In plain English

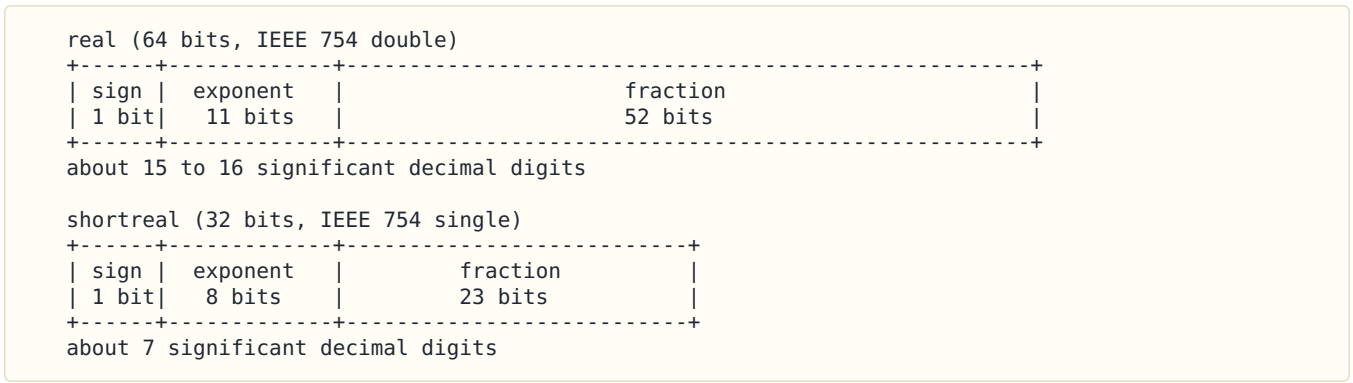
All the types so far hold whole numbers. A testbench also needs numbers with a fractional part: a clock period of 3.333 ns, a frequency of 2.4 GHz, a coverage figure of 87.5 percent, a voltage in an analogue model. These are stored in **real** (floating-point) types.

Technical definition

Type	Width	Equivalent C type	Default value	Use
`real`	64 bits	`double` (IEEE 754 double precision)	0.0	General floating-point numbers
`shortreal`	32 bits	`float` (IEEE 754 single precision)	0.0	Lower precision; mostly for DPI
`realtime`	64 bits	`double`	0.0	Exactly the same type as `real`; the name documents that the value is a time

Real types are not two-state or four-state. They are not built from individually accessible bits at all, so the concept does not apply: a real variable can never hold X or Z.

Memory representation



What you can and cannot do with a real

Allowed	Not allowed
`+`, `-`, `*`, `/`, `**`, unary `-`	`%` (modulus)
`<`, `<=`, `>`, `>=`, `==`, `!=`	`===`, `!==`, `==?`, `!=?`
`&&`, `||`, `!`, `?:`	Bitwise, reduction and shift operators
Assignment to and from integer types (with conversion)	Bit-select or part-select such as `r[3]`

Allowed	Not allowed
`$display` with `%f`, `%e`, `%g`	Use as an array index, or edge detection with `posedge`

Real types are **not synthesizable**.

Conversion between real and integer

Operation	Behaviour	3.7	2.5	-2.5
Implicit assignment `i = r;`	Rounds to nearest; a tie rounds away from zero	4	3	-3
Cast `int'(r)`	Same as implicit assignment: rounds	4	3	-3
`$rtoi(r)`	Truncates toward zero	3	2	-2
`$itor(i)`	Integer to real			
`$realtobits(r)`	The 64-bit IEEE pattern as a `bit [63:0]`, no numeric conversion			
`$bitstoreal(b)`	The reverse of `$realtobits`			
`$ceil(r)`, `$floor(r)`	Round up / round down, result is a real	4.0 / 3.0	3.0 / 2.0	-2.0 / -3.0

> **Interview trap:** In C, converting a `double` to an `int` truncates. In SystemVerilog, assigning a real to an integer **rounds**. Use `$rtoi` when you need truncation.

Complete example

tb_real.sv

```
module tb_real;

  real r;
  int  i;

  initial begin
    $display("1. default real      = %0f", r);

    r = 3.7;
    i = r;          $display("2. i = r      (3.7)  = %0d", i);
    i = int'(r);    $display("3. int'(r)    (3.7)  = %0d", i);
    i = $rtoi(r);   $display("4. $rtoi(r)   (3.7)  = %0d", i);
    i = int'(2.5); $display("5. int'(2.5)         = %0d", i);
    i = int'(-2.5);$display("6. int'(-2.5)        = %0d", i);

    r = 7 / 2;      $display("7. r = 7 / 2         = %0.2f", r);
    r = 7 / 2.0;    $display("8. r = 7 / 2.0       = %0.2f", r);

    r = 0.1 + 0.2;
    if (r == 0.3) $display("9. 0.1 + 0.2 == 0.3");
    else          $display("9. 0.1 + 0.2 != 0.3, it is %0.17f", r);

    if ((r - 0.3) < 1.0e-9 && (0.3 - r) < 1.0e-9)
      $display("10. equal within a tolerance of 1e-9");
  end

endmodule
```

EXPECTED OUTPUT

```
1. default real      = 0.000000
2. i = r      (3.7)  = 4
3. int'(r)    (3.7)  = 4
4. $rtoi(r)   (3.7)  = 3
5. int'(2.5)         = 3
6. int'(-2.5)        = -3
7. r = 7 / 2         = 3.00
8. r = 7 / 2.0       = 3.50
9. 0.1 + 0.2 != 0.3, it is 0.30000000000000004
10. equal within a tolerance of 1e-9
```

Line by line

Line	Explanation
1	A real starts at 0.0, not X.
2, 3	Implicit conversion and the `int'()` cast both round 3.7 to 4.
4	`$rtoi` discards the fraction: 3.
5, 6	A value exactly halfway is rounded away from zero: 2.5 → 3 and -2.5 → -3.
7	Both operands of `7 / 2` are integers, so integer division is performed first (result 3) and only then converted to real: 3.00. The type of the destination does not influence how the right-hand side is calculated.
8	One real operand makes the whole division real: 3.50.
9	0.1, 0.2 and 0.3 cannot be represented exactly in binary floating point, so the sum differs from 0.3 in the 17th digit.
10	The correct way to compare reals: check that the difference is smaller than a tolerance.

time, realtime, $time and $realtime

Simulation time is measured in units set by the **time unit** and rounded to the **time precision**. They are set with the `` `timescale `` directive or, preferably in SystemVerilog, with `timeunit` and `timeprecision` inside the module.

Item	What it is
`` `timescale 1ns/1ps ``	Time unit 1 ns (so `#1` means 1 ns); precision 1 ps (delays are rounded to the nearest ps)
`time`	64-bit unsigned four-state integer variable for storing times
`realtime`	A `real` variable for storing times with a fraction
`$time`	Returns the current time as a 64-bit integer, in time units, **rounded** to a whole unit
`$realtime`	Returns the current time as a real, in time units, including the fraction
`$stime`	Returns the low 32 bits of the current time
`$timeformat(unit, digits, suffix, width)`	Controls how `%t` prints times

tb_time.sv

```
`timescale 1ns/1ps

module tb_time;

  time     t_int;
  realtime t_real;
  realtime t_start, latency;

  initial begin
    $timeformat(-9, 3, " ns", 0);    // unit = ns, 3 decimals, suffix, no minimum width

    #1.75;
    t_int  = $time;
    t_real = $realtime;
    $display("$time     = %0d", t_int);
    $display("$realtime = %0.3f", t_real);
    $display("with %%t   = %t", $realtime);

    t_start = $realtime;
    #12.5;
    latency = $realtime - t_start;
    $display("latency   = %0.2f ns", latency);
  end

endmodule
```

EXPECTED OUTPUT

```
$time     = 2
$realtime = 1.750
with %t   = 1.750 ns
latency   = 12.50 ns
```

Explanation

Code	Explanation
`` `timescale 1ns/1ps ``	One time unit is 1 ns; the simulator keeps time to 1 ps.
`$timeformat(-9, 3, " ns", 0);`	`-9` means "print in units of 10^-9 s". Called once, usually at the start of the test.
`#1.75;`	A delay of 1.75 ns, which is exactly 1750 ps, so nothing is lost to rounding.
`t_int = $time;`	`$time` rounds 1.75 to the nearest whole time unit: 2. This is why `$time` can mislead you when delays have fractions.
`t_real = $realtime;`	Keeps the fraction: 1.750.
`latency = $realtime - t_start;`	The usual way to measure how long something took.

> **Common mistake:** With `` `timescale 1ns/100ps ``, the delay `#1.75` is rounded to the precision and becomes 1.8 ns. If a generated clock has the wrong frequency, check the time precision first.

Real-world verification uses of real

```
`timescale 1ns/1ps
module tb_clock_gen;
  logic clk = 0;
  real  freq_mhz  = 150.0;
  real  period_ns = 1000.0 / freq_mhz;      // 6.667 ns
  always #(period_ns / 2.0) clk = ~clk;
endmodule
```

- Clock generators with periods that are not whole numbers, as above.
- Measuring frequency, duty cycle, latency and throughput in monitors and checkers.
- Coverage: `$get_coverage()` and the `get_coverage()` method of a covergroup return a `real` percentage.
- Real-number modelling of analogue blocks (PLLs, ADCs, voltage regulators) in mixed-signal verification.
- Computing percentages and averages for end-of-test reports.

Interview questions

Q1. What is the difference between `real` and `shortreal`? `real` is 64-bit double precision (C `double`). `shortreal` is 32-bit single precision (C `float`).

Q2. What is the difference between `real` and `realtime`? None in behaviour. They are the same type. `realtime` is used for variables that hold time values, to make the code self-documenting.

Q3. What is the difference between `time` and `realtime`? `time` is a 64-bit unsigned four-state integer and holds a whole number of time units. `realtime` is a real and can hold fractional time.

Q4. What is the difference between `$time` and `$realtime`? `$time` returns an integer, rounded to the nearest whole time unit. `$realtime` returns a real number that includes the fractional part.

Q5. What is the default value of a `real`? 0.0.

Q6. `real r = 5 / 2;` What is `r`? 2.0. The division is done with integers because both operands are integers; the result 2 is then converted to real. Write `5.0 / 2` or `real'(5) / 2` to get 2.5.

Q7. `int i = 2.5;` What is `i`? 3. Real-to-integer conversion rounds, with ties away from zero. `$rtoi(2.5)` gives 2.

Q8. Are real types synthesizable? No. They are used in testbenches and in behavioural or real-number models.

Q9. Why should you not compare two reals with `==`? Because most decimal fractions cannot be represented exactly in binary floating point, so two values that are mathematically equal can differ in the last bits. Compare the absolute difference against a small tolerance instead.

Part 3: Packed, Unpacked and User-Defined Types

3.1 Packed and Unpacked Data Types

In plain English

Think of two ways of storing eight items.

- **Packed:** the eight items are glued together into one solid block. You can treat the block as a single number (add to it, shift it, compare it), and you can also pick out individual pieces.
- **Unpacked:** the eight items sit in eight separate boxes on a shelf. You work with one box at a time. The shelf as a whole is not a number.

Technical definition

- A **packed** type is a contiguous set of bits with a guaranteed bit order. It is an *integral* type: the whole thing can be used as one vector in arithmetic, logical, shift and comparison operations. Packed dimensions are written **before** the name.
- An **unpacked** type is a collection of separate elements. The simulator may store the elements however it likes. Only element-wise access and a few whole-aggregate operations (copy, compare) are defined. Unpacked dimensions are written **after** the name.

```
       logic [7:0]  data  [0:3];
             -----        -----
           packed         unpacked
         dimension        dimension
       (before the name)  (after the name)

  Read it as: "data is an unpacked array of 4 elements; each element is a packed 8-bit vector".
```

What can be packed

Can be packed	Cannot be packed (always unpacked)
Single-bit types with dimensions: `bit [..]`, `logic [..]`, `reg [..]`	`real`, `shortreal`, `realtime`
Packed arrays of packed arrays: `logic [3:0][7:0]`	`string`, `event`, `chandle`
`enum` types	Class handles
`struct packed`, `union packed` (members must themselves be packed)	Dynamic arrays, queues, associative arrays
`byte`, `shortint`, `int`, `longint`, `integer`, `time` are already packed vectors of fixed width	Unpacked structs and unions

Memory representation

The standard defines the layout of packed data exactly. It does not define the layout of unpacked data; the picture below shows the typical implementation, in which every unpacked element is stored in its own machine word or words.

```
Packed:   bit [3:0][7:0] p;          one 32-bit vector, contiguous

    bit:  31      24 23      16 15       8 7        0
          +----------+----------+----------+----------+
          |   p[3]   |   p[2]   |   p[1]   |   p[0]   |      1 word
          +----------+----------+----------+----------+

Unpacked: bit [7:0] u [4];           four separate elements

          +------------------------------+----------+
   u[0]   |          (unused)            |  8 bits  |      1 word
          +------------------------------+----------+
   u[1]   |          (unused)            |  8 bits  |      1 word
          +------------------------------+----------+
   u[2]   |          (unused)            |  8 bits  |      1 word
          +------------------------------+----------+
   u[3]   |          (unused)            |  8 bits  |      1 word
          +------------------------------+----------+
```

For four-state types the storage doubles (Section 2.1).

Comparison table

Property	Packed	Unpacked
Dimension position	Before the name: `logic [7:0] a;`	After the name: `logic a [8];`
Storage	Contiguous bits, defined order	Separate elements, layout chosen by the tool
Is it one number?	Yes (integral type)	No
Arithmetic on the whole (`a + 1`)	Yes	No
Bitwise / reduction / shift on the whole	Yes	No
Bit-select and part-select	Yes: `a[3]`, `a[7:4]`	Element select and slice: `a[3]`, `a[0:1]`
Assign from a number (`a = 8'hFF`)	Yes	No, needs an assignment pattern `'{...}`
Whole copy and `==` comparison	Yes	Yes, between arrays of the same type and size
Element types allowed	Only packed (bit-level) types	Any type: `int`, `real`, `string`, class handles, structs
Size fixed at compile time	Always	Fixed-size arrays yes; dynamic arrays, queues and associative arrays no
Print with	`%b`, `%h`, `%d`	`%p`, or loop over the elements
Synthesizable	Yes	Fixed-size arrays yes
Typical use	Buses, registers, bit fields	Memories, lists of transactions, lookup tables

Interview questions

Q1. What is the difference between packed and unpacked? A packed type is one contiguous vector of bits with a defined order that can be operated on as a single integral value. An unpacked type is a collection of independent elements that are accessed individually. Packed dimensions come before the name, unpacked dimensions after.

Q2. In `logic [7:0] mem [0:255];`, which dimension is packed? `[7:0]` is packed and `[0:255]` is unpacked. `mem` is an unpacked array of 256 elements, each a packed 8-bit vector. This is the standard way to model a memory.

Q3. Can you make a packed array of `real` or `string`? No. Packed types must be made of bits. `real`, `string`, class handles and events can only be elements of unpacked arrays.

3.2 Packed Arrays and Unpacked Arrays

This section introduces fixed-size arrays as data types. Module 2 covers dynamic arrays, associative arrays, queues and all the array methods in depth.

Packed arrays

A vector such as `logic [7:0]` is a one-dimensional packed array. Adding more packed dimensions subdivides the same vector into fields:

```
logic [31:0]       word_a;    // 32 bits
logic [3:0][7:0]   word_b;    // the same 32 bits, viewed as 4 bytes; word_b[3] is the top byte
logic [1:0][3:0][7:0] dword;  // 64 bits: 2 words of 4 bytes of 8 bits
```

The leftmost packed dimension is the outermost subdivision, and the first-declared index (the left one in `[3:0]`) is the most significant.

Unpacked arrays

```
int         a1 [0:7];          // 8 ints, indices 0 to 7
int         a2 [8];            // C-style shorthand, identical to [0:7]
logic [7:0] mem [0:255];       // 256 bytes
int         grid [4][3];       // 2-dimensional: 4 rows of 3 ints
```

Mixed arrays and index order

```
logic [3:0][7:0] lut [2][5];
//      |    |          |  |
//      3rd  4th        1st 2nd      order in which indices are written when accessing
```

Rule: when accessing an element, write the unpacked indices first (left to right), then the packed indices (left to right).

```
lut              the whole   array: 2 x 5 elements of 32 bits
lut[1]           one row  : 5 elements of 32 bits (unpacked array)
lut[1][4]        one element  : a 32-bit packed value
lut[1][4][2]     byte 2   of that element: 8 bits
lut[1][4][2][7]  bit 7   of that byte
```

Initialisation and assignment

Syntax	Used for	Example
`{a, b, c}`	Concatenation: builds one **packed** vector	`word_a = {8'hDE, 8'hAD, 8'hBE, 8'hEF};`

Syntax	Used for	Example
`{4{8'hFF}}`	Replication: packed	`word_a = {4{8'hFF}};`
`'{a, b, c}`	Assignment pattern: one value **per element** of an unpacked array or struct	`a2 = '{0, 1, 2, 3, 4, 5, 6, 7};`
`'{default: v}`	Every element gets `v`	`mem = '{default: 8'h00};`
`'{n{v}}`	Replication inside a pattern	`a2 = '{8{0}};`
`'{index: v, default: w}`	Selected elements by index	`a2 = '{0: 5, 7: 9, default: 0};`
`'0`, `'1`, `'x`, `'z`	Fill a **packed** value of any width	`word_a = '1;`

Interview trap: `{...}` and `'{...}` are different. `{a, b}` is a concatenation: for packed data it joins bits into one vector. `'{a, b}` is an assignment pattern: it supplies one value per element of an unpacked array, or per member of a struct. For a struct, for a multi-dimensional array, and whenever you need `default:` or index keys, only `'{...}` works. Since IEEE 1800-2009 a one-dimensional unpacked array may also be written as `{1, 2, 3}` (an "unpacked array concatenation", the form normally used with queues in Module 2), but the safe habit is: `{}` for packed vectors, `'{}` for unpacked arrays and structs.

Complete example

tb_packed_unpacked.sv

```
module tb_packed_unpacked;

  logic [3:0][7:0] word;       // packed: 4 bytes forming one 32-bit vector
  bit   [7:0]      mem [4];    // unpacked array of 4 packed bytes, indices 0 to 3
  bit   [7:0]      copy [4];

  initial begin
    word = 32'hDEAD_BEEF;
    $display("1. word        = %h", word);
    $display("2. word[3]     = %h   word[0] = %h", word[3], word[0]);
    $display("3. word[1:0]   = %h", word[1:0]);
    $display("4. word[2][7]  = %b", word[2][7]);
    $display("5. word + 1    = %h", word + 1);
    $display("6. $bits(word) = %0d", $bits(word));

    mem = '{8'h11, 8'h22, 8'h33, 8'h44};
    $display("7. mem[0] = %h   mem[3] = %h", mem[0], mem[3]);
    $display("8. mem[1][3:0] = %h", mem[1][3:0]);
    $display("9. $bits(mem) = %0d   $size(mem) = %0d", $bits(mem), $size(mem));

    copy = mem;                              // whole-array copy
    copy[2] = 8'h00;
    if (copy == mem) $display("10. copy == mem");
    else             $display("10. copy != mem (mem[2]=%h copy[2]=%h)", mem[2], copy[2]);

    mem = '{default: 8'hFF};
    foreach (mem[i]) $display("11. mem[%0d] = %h", i, mem[i]);
  end

endmodule
```

EXPECTED OUTPUT

```
1. word        = deadbeef
2. word[3]     = de   word[0] = ef
3. word[1:0]   = beef
4. word[2][7]  = 1
5. word + 1    = deadbef0
6. $bits(word) = 32
7. mem[0] = 11   mem[3] = 44
8. mem[1][3:0] = 2
9. $bits(mem) = 32   $size(mem) = 4
10. copy != mem (mem[2]=33 copy[2]=00)
11. mem[0] = ff
11. mem[1] = ff
11. mem[2] = ff
11. mem[3] = ff
```

Line by line

Line	Explanation
1	A packed array is assigned from a plain 32-bit number.
2	Index 3 is the most significant byte (`DE`); index 0 is the least significant (`EF`).
3	A slice of the first packed dimension: bytes 1 and 0 together, 16 bits. Note that `word[15:8]` would be wrong here: the first dimension of `word` only has indices 3 to 0.
4	Byte 2 is `AD` = `1010_1101` ; its bit 7 is 1.
5	Because `word` is packed, arithmetic on the whole value is legal.
7	`'{...}` assigns the listed values to indices 0, 1, 2, 3 in that order, because `mem [4]` means `[0:3]` .
8	Unpacked index first (`[1]` selects `22`), then a packed part-select (`[3:0]` selects the low nibble, `2`).
9	`$bits` counts all bits (4 × 8). `$size` returns the number of elements in the first dimension.
10	Unpacked arrays of the same type can be copied and compared as a whole. The copy is independent of the original.
11	`foreach` iterates over every index. Module 2 covers it in detail.

Reading and writing out of range

Situation	Result
Read with an out-of-range index, four-state element type	Returns X
Read with an out-of-range index, two-state element type	Returns 0
Read with an index containing X or Z	Returns X (four-state) or 0 (two-state)
Write with an out-of-range or unknown index	The write is ignored; nothing changes

Most simulators issue a warning, not an error, and continue.

> **Common mistake:** `logic [7:0] mem [256];` and `logic [0:7] mem [256];` are both legal but number the bits in opposite directions. `[7:0]` (little-endian bit numbering, MSB has the highest index) is the convention almost everywhere. Mixing the two in one project causes bit-reversal bugs.

> **Verification application:** A reference model of a RAM in a scoreboard is usually `logic [7:0] mem [int]` (an associative array, Module 2) for large sparse memories, or a fixed unpacked array `logic [7:0] mem [0:255]` for small ones. A register's bit fields are usually a packed struct (next section) so that the same 32 bits can be handled as a whole word on the bus and as named fields in the checker.

Interview questions

Q1. What is the difference between `logic [7:0] a;` and `logic a [7:0];`? The first is a packed 8-bit vector: one integral value that supports arithmetic and can be assigned a number. The second is an unpacked array of eight separate 1-bit elements: you cannot write `a = 8'hFF` or `a + 1`.

Q2. Declare a memory of 1024 locations, each 16 bits wide. `logic [15:0] mem [0:1023];` or `logic [15:0] mem [1024];`

Q3. For `bit [3:0][7:0] a [2][3];`, how many bits in total and what is `a[1][2][3]`? 2 × 3 × 4 × 8 = 192 bits. `a[1][2]` is one 32-bit element; `a[1][2][3]` is its most significant byte (8 bits).

Q4. What is the difference between `{}` and `'{}`? `{}` is the concatenation operator and produces one packed vector. `'{}` is an assignment pattern that provides one value per element of an unpacked array or per member of a struct.

Q5. What does `'1` mean? How is it different from `1` and `1'b1`? `'1` sets every bit of the target to 1, whatever its width. `1` is the 32-bit signed number one. `1'b1` is a single bit of value 1. For a 16-bit `x`: `x = '1` gives `FFFF`, while `x = 1` and `x = 1'b1` both give `0001`.

Q6. What happens when you read `mem[300]` from a 256-entry array? Not a fatal error. A four-state array returns X and a two-state array returns 0, normally with a simulator warning.

3.3 Structures (struct)

In plain English

An array groups values of the **same** type under one name. A structure groups values of **different** types under one name. A bus transfer has an address, some data and a read/write flag; a struct lets you carry all three around as one object called, say, `req`, and refer to the parts as `req.addr`, `req.data` and `req.write`.

Technical definition

A `struct` is a user-defined aggregate data type consisting of named **members** (also called fields), each with its own data type. A struct is **unpacked by default**. Adding the keyword `packed` makes it a packed struct, which is a single vector with named sub-fields.

Why it is needed

Without structs, related signals are separate variables. Passing them to a function means a long argument list, copying them means several assignments, and adding a field means editing every place they are used. A struct turns them into one unit that can be copied with one assignment, passed as one argument, and passed through a module port as one signal.

Syntax

```
// Unpacked struct (default): members may be of any type
typedef struct {
  int       id;
  bit [7:0] data;
  string    name;
  real      delay_ns;
} item_t;

// Packed struct: members must be packed types; the whole struct is one vector
typedef struct packed {
  logic [3:0] kind;     // most significant field  : bits [15:12]
  logic [3:0] len;      //                           bits [11:8]
  logic [7:0] addr;     // least significant field : bits [7:0]
} hdr_t;

item_t item;            // variables of the new types
hdr_t  hdr;
```

A struct is nearly always declared through `typedef`, so that the type has a name that can be reused for variables, ports and function arguments.

How it works internally

```
  Packed struct hdr_t  (16 bits, one vector)

      bit:  15      12 11       8 7                 0
           +----------+----------+-------------------+
           |   kind   |   len    |       addr        |
           +----------+----------+-------------------+
            first member = most significant bits

      hdr          is a 16-bit value   (hdr = 16'hA5C3 is legal)
      hdr.len      is bits [11:8]
      hdr[7:0]     is the same as hdr.addr

  Unpacked struct item_t  (separate members, no defined bit layout)

           +------------------+
           | id       : int   |
           +------------------+
           | data     : 8 bit |
           +------------------+
           | name     : string|
           +------------------+
           | delay_ns : real  |
           +------------------+

      item         is NOT a number; only item.<member> access, whole copy, and == / !=
```

Rule: in a packed struct, the **first** member declared occupies the **most significant** bits.

If any member of a packed struct is four-state, the whole struct is four-state. A packed struct is unsigned unless declared `struct packed signed`.

Packed versus unpacked struct

Property	Packed struct	Unpacked struct
Keyword	`struct packed`	`struct`
Member types	Only packed types (bits, vectors, enums, packed structs)	Any type (`int` , `real` , `string` , arrays, class handles)

Property	Packed struct	Unpacked struct
Is one vector	Yes	No
Assign from / to a vector	Yes: `hdr = 16'hA5C3;`	No
Bit-select / part-select on the whole	Yes: `hdr[7:0]`	No
Arithmetic on the whole	Yes	No
`$bits()`	Sum of member widths	Sum of member widths if all members are fixed-size
Member default values in the typedef	Not allowed	Allowed: `int id = 1;`
Synthesizable	Yes	Yes with synthesizable member types
Typical use	Protocol headers, register fields, instruction formats, struct-typed RTL ports	Grouping mixed-type testbench data, configuration records

Complete example

tb_struct.sv

```
module tb_struct;

  typedef struct packed {
    logic [3:0] kind;
    logic [3:0] len;
    logic [7:0] addr;
  } hdr_t;

  typedef struct {
    int       id;
    bit [7:0] data;
    string    name;
    real      delay_ns;
  } item_t;

  hdr_t  hdr;
  item_t item, item_copy;

  initial begin
    // ---------------- packed struct ----------------
    hdr = 16'hA5C3;
    $display("1. kind=%h len=%h addr=%h", hdr.kind, hdr.len, hdr.addr);

    hdr.len = 4'hF;
    $display("2. hdr=%h  hdr[7:0]=%h  $bits(hdr)=%0d", hdr, hdr[7:0], $bits(hdr));

    hdr = '{kind: 4'h1, len: 4'h2, addr: 8'h34};
    $display("3. hdr=%h", hdr);

    hdr = '{default: '0};
    $display("4. hdr=%h", hdr);

    // ---------------- unpacked struct ----------------
    item = '{id: 7, data: 8'hAB, name: "pkt", delay_ns: 1.5};
    item_copy    = item;        // copies every member
    item_copy.id = 8;
    $display("5. item.id=%0d  item_copy.id=%0d", item.id, item_copy.id);

    if (item == item_copy) $display("6. structs are equal");
    else                   $display("6. structs differ");

    $display("7. item = %p", item);
  end

endmodule
```

EXPECTED OUTPUT

```
1. kind=a len=5 addr=c3
2. hdr=afc3  hdr[7:0]=c3  $bits(hdr)=16
3. hdr=1234
4. hdr=0000
5. item.id=7  item_copy.id=8
6. structs differ
7. item = '{id:7, data:'hab, name:"pkt", delay_ns:1.5}
```

The exact text printed by `%p` on line 7 differs slightly between simulators (spacing, number format); the content is the same.

Line by line

Line	Explanation
`hdr = 16'hA5C3;`	A packed struct accepts a plain vector. `A` goes to `kind` (the first, most significant member), `5` to `len`, `C3` to `addr`.
`hdr.len = 4'hF;`	Writes one member. The vector becomes `AFC3`.
`hdr[7:0]`	A part-select on the struct as a vector, equal to `hdr.addr`.
`'{kind: ..., len: ..., addr: ...}`	Assignment pattern with member names. Order does not matter when names are given. Positional form `'{4'h1, 4'h2, 8'h34}` also works, in declaration order.
`'{default: '0}`	Every member is set to zero.
`item_copy = item;`	Assigning a struct copies all of its members. A struct variable holds the data itself (unlike a class handle, which holds a reference; see Module 4). Changing `item_copy.id` does not affect `item`.
`item == item_copy`	Unpacked structs of the same type can be compared member by member with `==` and `!=`.
`%p`	Prints an aggregate in assignment-pattern form. Very useful for debug.

Struct versus class (a preview of Module 4)

Property	`struct`	`class`
Nature	Static data container	Dynamic object
Memory allocation	When the variable is declared	When `new()` is called
A variable holds	The data itself	A handle (reference) to an object
Assignment `a = b`	Copies all the data	Copies only the handle; both names then refer to one object
Methods (functions, tasks)	Not allowed	Allowed
Constraints and `randomize()`	No `randomize()` method and no constraint blocks of its own. It can be randomized as a `rand` property of a class	Yes
Inheritance, polymorphism	No	Yes
Synthesizable	Yes	No
Typical use	RTL signal bundles, bit fields	Transactions and all testbench components

> **Common mistake:** Putting an `int`, `string` or `real` member in a packed struct. All members of a packed struct must be packed types. (`int` is allowed because it is a 32-bit packed type; `string` and `real` are not.)

> **Common mistake:** Forgetting the member order rule. If a 16-bit header arrives on a bus as `{kind, len, addr}`, the struct must declare `kind` first. Declaring the members in the reverse order silently swaps the fields.

> **Verification application:** RTL designers define bus request and response types as packed structs in a package. The testbench imports the same package, so the monitor can write `hdr_t h = hdr_t'(vif.data);` and read named fields, instead of using error-prone part-selects like `vif.data[11:8]`.

Interview questions

Q1. What is a struct? A user-defined type that groups several members of possibly different types under one name. Members are accessed with the dot operator.

Q2. What is the difference between a packed and an unpacked struct? A packed struct is stored as one contiguous vector with the first member in the most significant bits, so it can be assigned from a vector, sliced and used in arithmetic; all its members must be packed types. An unpacked struct is a collection of separate members of any type and supports only member access, whole copy and equality comparison. Structs are unpacked by default.

Q3. In a packed struct, which member is in the MSBs? The first one declared.

Q4. Is a struct synthesizable? Yes. Packed structs are fully synthesizable and widely used in RTL. Unpacked structs are synthesizable when their members are.

Q5. What is the difference between a struct and a class? A struct is a static grouping of data with no methods, and a struct variable contains the data. A class is created dynamically with `new`, is accessed through a handle, and supports methods, randomization with constraints, inheritance and polymorphism. Structs are synthesizable; classes are not.

Q6. How do you initialise all members of a struct to zero? `s = '{default: 0};` For a packed struct `s = '0;` also works.

3.4 Unions (union)

In plain English

A struct gives each member its own storage; all members exist side by side. A union gives all members the **same** storage; they are different ways of looking at the same bits. Writing through one member changes what you read through every other member.

Technical definition

A `union` is a user-defined type whose members all share one storage location. At any time it holds a single value, which can be read or written through any member.

- A **packed union** is one vector. All members must be packed types of **exactly the same width**. Reading through a different member than the one written is well defined: you see the same bits reinterpreted.
- An **unpacked union** may have members of any type and different sizes. The standard does not define the result of reading a member other than the one most recently written (except in special cases), so the behaviour depends on the simulator.
- A **tagged union** additionally stores a tag that records which member is currently valid, and the tool checks every access against the tag.

Why it is needed

The same bits often have several meaningful interpretations:

- A 32-bit instruction is a whole word on the bus, four bytes in a memory model, and `{opcode, operand}` to the decoder.
- A register is a 32-bit value on the bus and a set of named fields in the checker.
- A data channel carries a different packet format depending on a type field.

A packed union lets one variable be accessed in every one of these forms with no shifting, masking or casting.

Syntax

```
typedef union packed {
  logic [31:0]     word;                // view 1: one 32-bit value
  logic [3:0][7:0] bytes;               // view 2: four bytes
  struct packed {
    logic [7:0]  opcode;                // view 3: instruction fields
    logic [23:0] operand;
  } instr;
} reg_view_u;
```

How it works internally

```
  Packed union reg_view_u  (32 bits of storage, shared by all three members)

      bit:   31        24 23        16 15         8 7          0
           +------------+------------+------------+------------+
  word     |                      word[31:0]                    |
           +------------+------------+------------+------------+
  bytes    |  bytes[3]  |  bytes[2]  |  bytes[1]  |  bytes[0]  |
           +------------+------------+------------+------------+
  instr    |   opcode   |              operand                  |
           +------------+---------------------------------------+

   One set of bits. Three names for it.
   $bits(reg_view_u) = 32   (the width of one member, not the sum)

  For comparison, a struct with the same three members would occupy 32 + 32 + 32 = 96 bits.
```

Complete example

tb_union.sv

```
module tb_union;

  typedef union packed {
    logic [31:0]     word;
    logic [3:0][7:0] bytes;
    struct packed {
      logic [7:0]  opcode;
      logic [23:0] operand;
    } instr;
  } reg_view_u;

  reg_view_u u;

  initial begin
    u.word = 32'h12_345678;
    $display("1. word    = %h", u.word);
    $display("2. bytes   = %h %h %h %h", u.bytes[3], u.bytes[2], u.bytes[1], u.bytes[0]);
    $display("3. opcode  = %h   operand = %h", u.instr.opcode, u.instr.operand);

    u.bytes[0] = 8'hFF;                  // write through one view...
    $display("4. word    = %h", u.word); // ...and read through another

    u.instr.opcode = 8'hA0;
    $display("5. word    = %h", u.word);

    $display("6. $bits   = %0d", $bits(u));
  end

endmodule
```

EXPECTED OUTPUT

```
1. word    = 12345678
2. bytes   = 12 34 56 78
3. opcode  = 12   operand = 345678
4. word    = 123456ff
5. word    = a03456ff
6. $bits   = 32
```

Line by line

Line	Explanation
`u.word = 32'h12_345678;`	Writes all 32 shared bits.
Lines 2, 3	The same bits read as four bytes, and as an 8-bit opcode plus a 24-bit operand. Nothing was copied or converted.
`u.bytes[0] = 8'hFF;`	Changes only the lowest 8 of the shared bits. `u.word` shows the change because it is the same storage.
`u.instr.opcode = 8'hA0;`	Changes the top 8 bits.
`$bits(u)`	32: the size of the union is the size of one member, because they overlap.

Tagged unions (awareness level)

```
typedef union tagged {
  void        Invalid;          // a member with no data
  logic [7:0] Data;
} maybe_byte_t;

maybe_byte_t m;

initial begin
  m = tagged Data 8'h5A;        // stores the value and sets the tag to "Data"
  case (m) matches
    tagged Invalid : $display("no value");
    tagged Data .d : $display("value = %h", d);
  endcase
end
```

A tagged union is type-safe: reading a member that does not match the current tag is a run-time error. Support for tagged unions and pattern matching varies between tools, and they are rare in production code, so it is enough to know that they exist and what problem they solve.

Struct versus union

Property	`struct`	`union`
Storage	Each member has its own storage	All members share one storage
Total size	Sum of all member sizes	Size of one member (packed: all members equal)
Members valid at the same time	All	One value, seen through any member
Writing one member	Does not affect the other members	Changes what every other member reads
Purpose	Group different pieces of data together	View the same data in different formats
Packed form rule	Members must be packed types	Members must be packed types **of equal width**
Synthesizable	Yes	Packed unions yes

Common mistake: Declaring a packed union with members of different widths, for example `logic [31:0] word; logic [7:0] b;`. This is a compile error. Pad the narrow member inside a packed struct so the widths match.

Common mistake: Using an unpacked union to convert between types (for example writing an `int` member and reading a `shortreal` member). The result is not defined by the standard. Use a packed union, a cast, or `$realtobits` / `$bitstoreal`.

Verification application: A register model stores each register as a packed union of `logic [31:0] value` and a packed struct of fields. The bus monitor writes `reg.value = bus_data;` and the checker reads `reg.fields.enable`. One variable serves both the bus view and the functional view.

Interview questions

Q1. What is a union? A user-defined type in which all members share the same storage, so the same bits can be accessed under different names and formats.

Q2. What is the difference between a struct and a union? In a struct each member has separate storage and the total size is the sum of the members. In a union all members overlap, the size is that of one member, and writing one member changes all the others.

Q3. What is the restriction on a packed union? Every member must be a packed type and all members must have the same number of bits.

Q4. What is a tagged union? A union that also stores a tag identifying the currently valid member. Values are written with the `tagged` keyword and the simulator checks that reads use the matching member, which makes the union type-safe.

Q5. Give a practical use of a union. Viewing a register or instruction word both as a single vector and as named fields; viewing a word as bytes for a byte-oriented memory model; a data bus that carries different packet formats.

3.5 Enumerated Data Types (enum)

In plain English

Hardware is full of small numbers that stand for something: state 0 means idle, state 1 means fetch; opcode 3 means OR. Writing the raw numbers makes code and waveforms unreadable, and nothing stops you from assigning state 7 when only four states exist. An **enumeration** gives each legal value a name and makes the compiler enforce that only those names are assigned.

Technical definition

An `enum` declares a data type whose legal values are a set of **named constants**. Each name has an integral value of an underlying **base type**. If no base type is given it is `int`. If no values are given, the first name is 0 and each following name is one more than the previous one.

Why it is needed

Without enum	With enum
`parameter IDLE = 2'b00, FETCH = 2'b01;` and `logic [1:0] state;`	`typedef enum logic [1:0] {IDLE, FETCH, EXEC, DONE} state_e;` and `state_e state;`
Waveform shows `2'b01`	Waveform shows `FETCH`
`state = 2'b11;` or any other number is accepted	Assigning a plain number is a compile error
`$display` prints a number	`state.name()` prints `"FETCH"`
Adding a state means renumbering by hand	Values are assigned automatically
Coverage bins and random values must be listed by hand	Coverage creates one bin per name; `randomize()` generates only named values

Syntax

```
// 1. Simplest form: base type int, values 0, 1, 2
enum {RED, GREEN, BLUE} light;                          // an anonymous enum variable

// 2. With typedef (the normal way) and an explicit base type
typedef enum logic [1:0] {IDLE, FETCH, EXEC, DONE} state_e;

// 3. With explicit values
typedef enum bit [2:0] {RD = 3'b001, WR = 3'b010, RST = 3'b100} cmd_e;

// 4. Mixed: unassigned names continue from the previous value
typedef enum {A = 5, B, C, D = 20, E} mix_e;          // A=5 B=6 C=7 D=20 E=21

// 5. Ranges of names
typedef enum {S[3]}        r1_e;     // S0, S1, S2          = 0, 1, 2
typedef enum {T[2:4]}      r2_e;     // T2, T3, T4          = 0, 1, 2
typedef enum {U[2] = 8}    r3_e;     // U0 = 8, U1 = 9
```

Rules for values

Rule	Example of a violation
Every name must have a unique value	`enum {A = 1, B, C = 2}`: `B` is automatically 2, which collides with `C` → error
Every value must fit in the base type	`enum bit [1:0] {A, B, C, D, E}`: `E` would be 4, which needs 3 bits → error
X or Z values are allowed only with a four-state base type, and a name following an X/Z value must be given an explicit value	`enum bit [1:0] {A = 2'bxx}` → error
Names must be unique within their scope	Two enums in the same module both containing `IDLE` → error. Use prefixes, or put the types in different packages or classes

How it works internally

An enum variable is stored exactly like a variable of its base type; `state_e` above is two four-state bits. The names are constants known to the compiler and to the debugger. The default value of an enum variable is the default of the base type: **0 for a two-state base, X for a four-state base**, whether or not that value has a name.

```
typedef enum logic [1:0] {IDLE, FETCH, EXEC, DONE} state_e;

   name      value      stored bits
   -----     -----      -----------
   IDLE        0            00
   FETCH       1            01
   EXEC        2            10
   DONE        3            11
```

Built-in methods

Method	Returns
`first()`	The first member of the enumeration
`last()`	The last member
`next(N)`	The Nth next member (default N = 1). Wraps around from the last to the first

Method	Returns
`prev(N)`	The Nth previous member (default N = 1). Wraps around from the first to the last
`num()`	The number of members
`name()`	The name of the current value as a `string`. Returns an empty string `""` if the value matches no member

Type-checking rules

Operation	Legal?	Explanation
`state = FETCH;`	Yes	Assigning a member of the same enum
`state = other_state;`	Yes	Same enum type
`int i = state;`	Yes	Enum → integer conversion is automatic
`i = state + 1;`	Yes	In an expression the enum is used as its base-type value; the result is an integer
`state = 1;`	**No**	Integer → enum needs a cast
`state = state + 1;`	**No**	The right-hand side is an integer
`state++;`	**No** (strictly)	Same reason; some simulators accept it with a warning
`state = state_e'(1);`	Yes	Static cast; **no check** that the value is a legal member
`$cast(state, 1);`	Yes	Dynamic cast; **checks** at run time that the value is a legal member
`state = state.next();`	Yes	The proper way to step to the next member

Complete example

`tb_enum.sv`

```
module tb_enum;

  typedef enum logic [1:0] {IDLE, FETCH, EXEC, DONE} state_e;
  typedef enum {RED = 1, GREEN = 2, BLUE = 4}        color_e;

  state_e state;
  color_e color;
  int     raw;

  initial begin
    $display("1. default state = %b, name = '%s'", state, state.name());
    $display("2. default color = %0d, name = '%s'", color, color.name());

    $display("3. state_e has %0d members:", state.num());
    state = state.first();
    do begin
      $display("     %-5s = %0d", state.name(), state);
      state = state.next();
    end while (state != state.first());

    state = DONE;
    state = state.next();
    $display("4. next of DONE    = %s", state.name());
    state = state.prev(2);
    $display("5. prev(2) of IDLE = %s", state.name());

    raw = 2;
    if ($cast(color, raw))  $display("6. $cast of 2 succeeded: %s", color.name());
    raw = 3;
    if (!$cast(color, raw)) $display("7. $cast of 3 failed, color is still %s", color.name());

    color = color_e'(raw);
    $display("8. static cast of 3: value = %0d, name = '%s'", color, color.name());

    color = BLUE;
    raw   = color + 1;
    $display("9. BLUE + 1 = %0d", raw);
  end

endmodule
```

EXPECTED OUTPUT

```
1. default state = xx, name = ''
2. default color = 0, name = ''
3. state_e has 4 members:
     IDLE  = 0
     FETCH = 1
     EXEC  = 2
     DONE  = 3
4. next of DONE    = IDLE
5. prev(2) of IDLE = EXEC
6. $cast of 2 succeeded: GREEN
7. $cast of 3 failed, color is still GREEN
8. static cast of 3: value = 3, name = ''
9. BLUE + 1 = 5
```

Line by line

Line	Explanation
1	The base type is `logic [1:0]` (four-state), so the variable starts as `xx`. X matches no member, so `name()` returns an empty string.

Line	Explanation
2	The base type is `int`, so the variable starts at 0. But 0 is not a member of `color_e` (the members are 1, 2 and 4). The variable holds an illegal value and `name()` returns an empty string. This is a classic trap.
3	The `do` ... `while` loop visits every member exactly once: start at `first()`, step with `next()`, stop when `next()` has wrapped around to `first()` again.
4	`next()` wraps from the last member to the first.
5	`prev(2)` steps back twice: `IDLE` → `DONE` → `EXEC`.
6	`$cast` used as a function returns 1 on success and performs the assignment. 2 is the value of `GREEN`.
7	3 is not a member. `$cast` returns 0 and leaves `color` unchanged.
8	The static cast does no checking. `color` now holds 3, an out-of-range value with no name.
9	Using an enum in arithmetic gives a plain integer (4 + 1 = 5), which may be stored in an `int` but not back into the enum without a cast.

Interview trap: How do you loop over all members of an enum? The obvious loop `for (state = state.first(); state <= state.last(); state = state.next())` never terminates. After the last member, `next()` wraps to the first, so the condition is always true. Use the `do` ... `while` loop from the example, or loop a counter from 0 to `num()-1` and call `next()` inside.

Interview trap: "What is the default value of an enum variable?" It is the default of the base type (0 for `int`), **not** the first member. If the first member is not 0, the variable starts with an illegal value. Good practice: let the first (reset/idle) member have the value 0.

Enums in RTL: a state machine

```
typedef enum logic [1:0] {IDLE, FETCH, EXEC, DONE} state_e;
state_e state, next_state;

always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) state <= IDLE;
  else        state <= next_state;

always_comb begin
  next_state = state;                       // default: stay
  unique case (state)
    IDLE : if (start) next_state = FETCH;
    FETCH:            next_state = EXEC;
    EXEC :            next_state = DONE;
    DONE :            next_state = IDLE;
  endcase
end
```

Enums are synthesizable. The waveform viewer shows `IDLE`, `FETCH` ... instead of numbers, which is a large practical benefit when debugging.

Verification application: In the ALU testbench the opcode field of the transaction is `rand alu_op_e op;`. The solver then generates only the eight legal opcodes. A coverpoint on `op` automatically gets one bin per name. Log messages print `op.name()`. The scoreboard's reference model is a `case (op)` on the names.

Interview questions

Q1. What is an enum and why is it used? A data type with a fixed set of named values. It makes code and waveforms readable and adds type safety: only declared names may be assigned without a cast.

Q2. What is the default base type and what are the default values? The base type is `int`. The first name is 0 and each following name is the previous value plus one.

Q3. What is the default value of an enum variable? The default value of its base type: 0 for a two-state base, X for a four-state base. It is not necessarily a legal member.

Q4. What are the enum methods? `first()`, `last()`, `next()`, `prev()`, `num()` and `name()`.

Q5. Can you assign an integer to an enum variable? Not directly; it is a compile error. Use a static cast `state_e'(value)`, which does not check the value, or `$cast(state, value)`, which checks at run time that the value is a member.

Q6. What does `name()` return for a value that is not a member? An empty string.

Q7. `typedef enum {A = 1, B, C = 2} t;` Is this legal? No. `B` automatically gets the value 2, and `C` is also 2. Enum values must be unique.

Q8. `typedef enum bit [1:0] {A, B, C, D, E} t;` Is this legal? No. Five members need values 0 to 4, and 4 does not fit in 2 bits.

Q9. How do you print the name of an enum value? `$display("%s", var.name());`. `%p` also prints the name in most simulators. `%d` prints the numeric value.

Q10. Can two enums in the same scope share a member name? No, names must be unique in a scope. Give the names a prefix (`ALU_ADD`, `BUS_IDLE`) or declare the enums in different packages or classes.

3.6 typedef

In plain English

`typedef` gives a new name to a type. It does not create any storage. After `typedef logic [7:0] byte_t;` you can write `byte_t x;` wherever you would have written `logic [7:0] x;`.

Why it is needed

1. **Readability and single point of change.** If the address width is defined once as `typedef logic [31:0] addr_t;`, changing to 64 bits is a one-line edit.
2. **Naming complex types.** Enums, structs and unions need a name to be reused for several variables, ports and arguments.
3. **Where the language requires a simple type name.** A function cannot directly return an unpacked array type, and a cast needs a type name; both are solved by a `typedef`.
4. **Forward declaration.** Two classes that refer to each other need one of them to be announced before it is defined.
5. **Short names for parameterized classes.** `typedef mailbox #(alu_txn) txn_mbx_t;`

Syntax and forms

```
// 1. Alias for a built-in type
typedef logic [7:0]  byte_t;
typedef int unsigned uint_t;

// 2. Enum, struct and union types (seen in the previous sections)
typedef enum logic [1:0] {IDLE, FETCH, EXEC, DONE} state_e;
typedef struct packed { logic [3:0] kind; logic [3:0] len; logic [7:0] addr; } hdr_t;

// 3. Array types: the dimension goes after the new type name, as in a variable declaration
typedef byte_t     mem_t  [0:255];     // unpacked array type: 256 bytes
typedef bit [7:0]  byte_q_t [$];       // queue-of-bytes type        (Module 2)
typedef int        int_da_t [];        // dynamic-array-of-int type  (Module 2)

// 4. Forward declaration of a class                              (Module 4)
typedef class driver;

// 5. Alias for a specialisation of a parameterized class         (Modules 7 and 11)
// typedef mailbox #(alu_txn) txn_mbx_t;
```

Complete example

tb_typedef.sv

```
module tb_typedef;

  typedef logic [7:0] byte_t;
  typedef byte_t      buf_t [4];                  // array type: 4 bytes
  typedef enum bit {READ, WRITE} dir_e;
  typedef struct {
    dir_e  dir;
    byte_t addr;
    buf_t  payload;
  } xfer_t;

  // A function can return an array only through a typedef name
  function automatic buf_t make_ramp(byte_t start);
    buf_t result;
    foreach (result[i]) result[i] = start + i;
    return result;
  endfunction

  xfer_t x;

  initial begin
    x.dir     = WRITE;
    x.addr    = 8'h40;
    x.payload = make_ramp(8'h10);
    $display("dir=%s addr=%h", x.dir.name(), x.addr);
    foreach (x.payload[i]) $display("payload[%0d] = %h", i, x.payload[i]);
    $display("$bits(byte_t) = %0d, $bits(buf_t) = %0d", $bits(byte_t), $bits(buf_t));
  end

endmodule
```

EXPECTED OUTPUT

```
dir=WRITE addr=40
payload[0] = 10
payload[1] = 11
payload[2] = 12
payload[3] = 13
$bits(byte_t) = 8, $bits(buf_t) = 32
```

Line by line

Line	Explanation
`typedef logic [7:0] byte_t;`	A name for an 8-bit four-state vector.
`typedef byte_t buf_t [4];`	A name for "unpacked array of 4 `byte_t`". Types can be built from other user-defined types.
`typedef enum bit {READ, WRITE} dir_e;`	A 1-bit enum: `READ` = 0, `WRITE` = 1.
`typedef struct {...} xfer_t;`	A struct type using the three types above.
`function automatic buf_t make_ramp(...)`	The return type is an array. Writing `function logic [7:0] [4] ...` is not possible; the typedef makes it legal. Functions are covered in Module 3.
`x.payload = make_ramp(8'h10);`	The returned array is copied into the struct member.
`$bits(byte_t)`	`$bits` accepts a type name as well as a variable.

typedef compared with related constructs

Construct	Creates	Example	Notes
`typedef`	A new name for a type	`typedef logic [7:0] byte_t;`	Scoped and type-checked. No storage
`` `define ``	A text substitution	`` `define BYTE logic [7:0] ``	Global, no scope, no checking, hard to debug. Avoid for types
`parameter` / `localparam`	A named constant **value**	`parameter int WIDTH = 8;`	Names a value, not a type
`parameter type`	A type that can be overridden per instance	`module fifo #(parameter type T = logic [7:0])`	Makes a module or class generic (Module 11)

Naming conventions used in industry: `_t` for general types, `_e` for enums, `_s` for structs and `_u` for unions. Typedefs shared between files are placed in a package.

> **Common mistake:** Declaring the same `typedef` in two modules and then connecting the two modules through a port of that type. Two separately declared structs or enums are *different types* even if they are written identically, and tools may reject the connection. Declare the type once in a package and import it in both places.

Interview questions

Q1. What is `typedef`? A statement that creates a new name for an existing or user-defined data type. It allocates no storage.

Q2. Why use `typedef` instead of `` `define ``? A `typedef` is a real type known to the compiler: it obeys scope rules, can be placed in a package, is type-checked, and works with casting and `$bits`. A `` `define `` is blind text replacement with global visibility.

Q3. What is a forward typedef and when is it needed? `typedef class B;` tells the compiler that `B` is a class that will be defined later. It is needed when class `A` contains a handle of class `B` and class `B` contains a handle of class `A`, or simply when a class is used before its definition appears in the file.

Q4. How do you write a function that returns an array? Create a `typedef` for the array type and use that name as the function's return type.

Q5. Where should shared typedefs be placed? In a package, which is then imported by every module, interface and class that needs them.

Part 4: Casting, Default Values, Four-State Logic and Operators

4.1 Type Casting: Static and Dynamic

In plain English

Casting means converting a value from one data type to another: a real number to an integer, a wide vector to a narrow one, an unsigned value to a signed one, a plain number to an enum. Sometimes SystemVerilog converts automatically (**implicit** conversion). Sometimes you must ask for it, or you want to make it visible to the reader (**explicit** cast).

There are two kinds of explicit cast:

- A **static cast** is decided at compile time and performs no run-time check. You are telling the compiler "trust me".
- A **dynamic cast**, `$cast`, checks at run time whether the value is legal for the destination type and tells you whether it succeeded.

Why casting is needed

- SystemVerilog is strongly typed in several places: an integer cannot be assigned to an enum, and a base-class handle cannot be assigned to a derived-class handle, without a cast.
- Implicit conversions can silently truncate, extend or reinterpret a value. An explicit cast documents that the conversion is intended and controls exactly how it happens.
- Linting tools flag implicit width and sign conversions in RTL; casts remove the warnings.

Implicit conversions (what happens when you do not cast)

Assignment	What happens automatically
Wider → narrower (`logic [3:0] n = 8'hAB;`)	Upper bits are discarded: `n` = `B`. No error
Narrower → wider, source unsigned	Zero-extended
Narrower → wider, source signed	Sign-extended
Four-state → two-state	X and Z become 0
Two-state → four-state	Value kept
Real → integer	Rounded to nearest, ties away from zero
Integer → real	Converted to the nearest representable real
Enum → integer	The value of the enum member
Integer → enum	**Not allowed** without a cast
String literal → packed vector	8 bits per character, right-justified

Static casting

Syntax: `<type>'(<expression>)`. Note the apostrophe between the type and the parenthesis.

Form	Meaning	Example	Result
`type'(expr)`	Convert to a named type	`int'(2.4)`	2
`N'(expr)`	Change the width to N bits	`4'(8'hAB)`	`4'hB`
`signed'(expr)`	Treat as signed	`signed'(4'b1111)`	-1
`unsigned'(expr)`	Treat as unsigned	`unsigned'(-1)`	4,294,967,295
`enum_type'(expr)`	Integer to enum, **unchecked**	`state_e'(2)`	`EXEC`
`struct_type'(expr)`	Vector to packed struct	`hdr_t'(16'hA5C3)`	Fields filled from the bits
`const'(expr)`	Treat as constant	`const'(x)`	Used mainly in assertions

The type in a static cast must be a simple type name. To cast to something like `logic [7:0]`, either use the size cast `8'(expr)` or define a `typedef` first.

Dynamic casting with $cast

Syntax: `$cast(destination, source)`

`$cast` can be called in two ways, and the difference is a frequent interview question.

Called as	On success	On failure
Function: `if ($cast(dst, src)) ...`	Assigns and returns 1	Returns 0, leaves `dst` unchanged, **no error message**
Task: `$cast(dst, src);`	Assigns	**Run-time error**, `dst` unchanged

`$cast` has two main uses:

1. **Integer → enum**, with a check that the value is a declared member (shown below).
2. **Base-class handle → derived-class handle** (downcasting), with a check that the object really is of the derived type. This is the more important use and is covered in Module 4.

How it works internally

```
Static cast   state = state_e'(raw);
-----------------------------------
compile time :  the compiler generates a plain copy (with resize / reinterpretation)
run time     :  bits are copied. No check. An illegal value is stored silently.

Dynamic cast  ok = $cast(state, raw);
-----------------------------------
compile time :  the compiler checks only that the two types could be compatible
run time     :  1. evaluate raw
                2. is the value a member of state_e ?
                     yes -> assign, return 1
                     no  -> do not assign, return 0   (task form: report an error)
```

Complete example

tb_casting.sv

```
module tb_casting;

  typedef enum bit [1:0] {OP_ADD, OP_SUB, OP_MUL, OP_DIV} op_e;
  typedef enum {RD = 1, WR = 2}                           dir_e;
  typedef struct packed {
    logic [3:0] hi;
    logic [3:0] lo;
  } nibbles_t;

  int         i;
  real        r;
  byte        b;
  logic [3:0] n;
  nibbles_t   nb;
  op_e        op;
  dir_e       dir;

  initial begin
    // ---- implicit conversions ----
    r = 3.7;
    i = r;                    $display("1.  i = 3.7              -> %0d", i);
    n = 8'hAB;                $display("2.  4-bit n = 8'hAB      -> %h", n);
    i = n;                    $display("3.  int = n              -> %0d", i);

    // ---- static casts ----
    i  = int'(2.4);           $display("4.  int'(2.4)            -> %0d", i);
    b  = byte'(300);          $display("5.  byte'(300)           -> %0d", b);
    n  = 4'(8'hAB);           $display("6.  4'(8'hAB)            -> %h", n);
    i  = signed'(4'b1111);    $display("7.  signed'(4'b1111)     -> %0d", i);
    nb = nibbles_t'(8'h5C);   $display("8.  nibbles_t'(8'h5C)    -> hi=%h lo=%h", nb.hi, nb.lo);
    op = op_e'(2);            $display("9.  op_e'(2)             -> %s", op.name());
    r  = real'(7) / 2;        $display("10. real'(7) / 2         -> %0.1f", r);

    // ---- dynamic casts ----
    if ($cast(dir, 2))        $display("11. $cast(dir, 2) ok     -> %s", dir.name());
    if (!$cast(dir, 5))       $display("12. $cast(dir, 5) failed -> dir stays %s", dir.name());

    dir = dir_e'(5);          $display("13. dir_e'(5)            -> value %0d, name '%s'",
                                       dir, dir.name());
    // $cast(dir, 7);         // task form: this line would produce a run-time error
  end

endmodule
```

EXPECTED OUTPUT

```
1.  i = 3.7              -> 4
2.  4-bit n = 8'hAB      -> b
3.  int = n              -> 11
4.  int'(2.4)            -> 2
5.  byte'(300)           -> 44
6.  4'(8'hAB)            -> b
7.  signed'(4'b1111)     -> -1
8.  nibbles_t'(8'h5C)    -> hi=5 lo=c
9.  op_e'(2)             -> OP_MUL
10. real'(7) / 2         -> 3.5
11. $cast(dir, 2) ok     -> WR
12. $cast(dir, 5) failed -> dir stays WR
13. dir_e'(5)            -> value 5, name ''
```

Line by line

Line	Explanation
1	Implicit real-to-integer conversion rounds 3.7 to 4.
2	Implicit truncation: only the low 4 bits of `AB` are kept.
3	`n` is unsigned, so it is zero-extended: `B` = 11.
4	2.4 rounds to 2.
5	300 is `1_0010_1100`. Only the low 8 bits `0010_1100` are kept: 44.
6	Size cast: the same truncation as line 2, but written explicitly so that a reader and a lint tool know it is intentional.
7	`4'b1111` is unsigned (15). `signed'` reinterprets it as -1, which is then sign-extended to 32 bits.
8	An 8-bit vector is cast to a packed struct: upper nibble to `hi`, lower nibble to `lo`.
9	Integer to enum by static cast. 2 is a legal member, `OP_MUL`.
10	The cast makes the left operand real, so the division is a real division. Compare with `7 / 2` = 3 in Section 2.7.
11	Function form of `$cast`; 2 is the value of `WR`, so it succeeds.
12	5 is not a member of `dir_e`. The function returns 0 and `dir` is not modified.
13	The static cast stores 5 anyway. `dir` now holds a value that is not a member; `name()` returns an empty string. This is the danger of static casts to enums.

Bit-stream casting and streaming operators

Two further forms convert between aggregates (unpacked structs, arrays) and flat bit vectors. They are used to **pack** a transaction into bits for driving, and to **unpack** bits back into fields in a monitor.

tb_bitstream.sv

```
module tb_bitstream;

  typedef struct {            // unpacked struct, 32 bits in total
    bit [7:0]  addr;
    bit [7:0]  data;
    bit [15:0] crc;
  } pkt_t;
  typedef bit [31:0] u32_t;

  pkt_t     p, q;
  u32_t     v;
  bit [7:0] b8;

  initial begin
    p = '{addr: 8'hA1, data: 8'hB2, crc: 16'hC3D4};

    v = u32_t'(p);                $display("1. bit-stream cast       : %h", v);
    v = {>>{p}};                  $display("2. streaming {>>{p}}     : %h", v);

    q = pkt_t'(32'h1122_3344);    $display("3. unpack                : addr=%h data=%h crc=%h",
                                           q.addr, q.data, q.crc);

    v  = {<<8{32'hA1B2_C3D4}};    $display("4. byte reverse {<<8{..}} : %h", v);
    b8 = {<<{8'b1100_0001}};      $display("5. bit reverse  {<<{..}}  : %b", b8);
  end

endmodule
```

EXPECTED OUTPUT

```
1. bit-stream cast       : a1b2c3d4
2. streaming {>>{p}}     : a1b2c3d4
3. unpack                : addr=11 data=22 crc=3344
4. byte reverse {<<8{..}} : d4c3b2a1
5. bit reverse  {<<{..}}  : 10000011
```

Line	Explanation
1	A bit-stream cast flattens the struct into a vector. The first member becomes the most significant bits. Source and destination must have the same total number of bits.
2	The streaming operator `{>>{...}}` does the same: it streams the members left to right into a vector.
3	The reverse direction: a 32-bit value is distributed over the members.
4	`{<<8{...}}` streams right to left in chunks of 8 bits, so the byte order is reversed. Used for endianness conversion.
5	`{<<{...}}` with no chunk size reverses bit by bit.

Static cast versus dynamic cast

Property	Static cast `type'(expr)`	Dynamic cast `$cast(dst, src)`
When it is resolved	Compile time	Run time
Checks whether the value is legal	No	Yes
On an illegal value	Silently stores it	Function form returns 0; task form gives a run-time error
Returns a status	No	Yes (function form)
Run-time cost	None	Small
Main uses	Width, sign, real↔integer, vector↔packed struct	Integer→enum, base-class handle→derived-class handle
Synthesizable	Yes	No (`$cast` is a simulation system task)

Interview trap: "What is the difference between calling `$cast` as a function and as a task?" As a function it returns 1 or 0 and never reports an error, so you must test the return value. As a task it reports a run-time error when the cast fails. Calling the function form and ignoring the result hides failures.

Common mistake: Using a static cast to put a value from the DUT into an enum, for example `op = alu_op_e'(vif.op);` in a monitor. If the DUT drives an illegal encoding, the enum silently holds a non-member value. Use `if (!$cast(op, vif.op)) $error("illegal opcode %0h", vif.op);`.

Common mistake: Forgetting the apostrophe. `int(x)` is a syntax error; the cast is `int'(x)`.

Interview questions

Q1. What is type casting? Converting a value of one data type into another data type, either implicitly by the language rules or explicitly with a cast operator or `$cast`.

Q2. What is the difference between static and dynamic casting? A static cast, `type'(expr)`, is resolved at compile time and does not check the value. A dynamic cast, `$cast(dst, src)`, checks at run time that the source value is valid for the destination type, assigns only if it is, and reports success or failure.

Q3. When must `$cast` be used? When assigning an integral value to an enum safely, and when assigning a base-class handle to a derived-class handle (downcasting).

Q4. What does `$cast` return? When called as a function it returns 1 if the cast succeeded and 0 if it failed. When called as a task there is no return value; a failure causes a run-time error.

Q5. What is `int'(3.5)`? What is `$rtoi(3.5)`? `int'(3.5)` is 4, because the cast rounds. `$rtoi(3.5)` is 3, because it truncates.

Q6. How do you convert a real to its 64-bit pattern? With `$realtobits(r)`. A cast `longint'(r)` would instead convert the numeric value (rounding it).

Q7. What does a size cast such as `16'(x)` do? It changes the width of the expression to 16 bits: truncating the upper bits if `x` is wider, or extending (sign- or zero-extending according to the signedness of `x`) if it is narrower.

4.2 Default Values of Different Data Types

The rule

A variable that has not been assigned has the default value of its data type:

- **Four-state** integral types start as **X**.
- **Two-state** integral types start as **0**.
- **Nets** have no stored value: an undriven net is **Z**.
- Everything dynamic starts **empty** or **null**.

Complete table

Data type	Default value	Notes
`bit`, `bit [N:0]`	0	Two-state
`byte`, `shortint`, `int`, `longint`	0	Two-state
`logic`, `reg`, `logic [N:0]`	X	Four-state
`integer`	X	Four-state
`time`	X	Four-state
`real`, `shortreal`, `realtime`	0.0	
`string`	`""` (empty string)	Length 0

Data type	Default value	Notes
`enum`	Default of the base type	0 for `int` or `bit` base; X for a `logic` base. Not necessarily a declared member
`wire`, `tri` and other nets	Z	When there is no driver
`trireg`	X	Charge-storage net
`tri0` / `tri1`	0 / 1	When there is no driver
Packed struct, packed union	Per bit: X if four-state, 0 if two-state	
Unpacked struct	Each member gets the default of its own type	Unless the typedef gives a member default
Fixed-size array	Each element gets the default of the element type	
Dynamic array	Empty, `size()` is 0	
Queue	Empty, `size()` is 0	
Associative array	Empty, `num()` is 0	
Class handle	`null`	No object exists until `new()` is called
Virtual interface	`null`	Must be assigned before use
`event`	A new, untriggered event	
`chandle` (C pointer for DPI)	`null`	
`mailbox`, `semaphore` variables	`null`	They are class handles; `new()` is required

Complete example

tb_defaults.sv

```
module tb_defaults;

  typedef enum {LOW, HIGH} level_e;

  class dummy;
  endclass

  bit       v_bit;
  logic     v_logic;
  int       v_int;
  integer   v_integer;
  real      v_real;
  string    v_string;
  wire      v_wire;
  level_e   v_enum;
  int       v_dyn [];
  int       v_queue [$];
  int       v_assoc [string];
  dummy     v_handle;
  struct { int a; logic [3:0] b; } v_struct;

  initial begin
    $display("bit=%b logic=%b int=%0d integer=%0d real=%0.1f",
             v_bit, v_logic, v_int, v_integer, v_real);
    $display("string='%s' (length %0d)", v_string, v_string.len());
    $display("wire=%b", v_wire);
    $display("enum=%s", v_enum.name());
    $display("dynamic array size=%0d  queue size=%0d  associative array entries=%0d",
             v_dyn.size(), v_queue.size(), v_assoc.num());
    if (v_handle == null) $display("class handle is null");
    $display("struct: a=%0d b=%b", v_struct.a, v_struct.b);
  end

endmodule
```

EXPECTED OUTPUT

```
bit=0 logic=x int=0 integer=x real=0.0
string='' (length 0)
wire=z
enum=LOW
dynamic array size=0  queue size=0  associative array entries=0
class handle is null
struct: a=0 b=xxxx
```

Related rules

Situation	Value
Declaration with an initialiser, `int count = 5;` at module level	Set once, before time 0 processes run
Variable declared `static` (the default inside a module-level task or function) with an initialiser	Initialised once at the start of simulation, **not** on each call. A frequent bug; see Module 3
`automatic` variable with an initialiser	Initialised on every entry to the task, function or block
Reading a non-existent associative array entry	Returns the default value of the element type, with a warning
Reading out of range from an array	X for four-state elements, 0 for two-state

Situation	Value
`rand` class property before `randomize()` is called	The normal default of its type (0 for `bit` / `int`)
Function return value when no `return` is executed	The default of the return type

Interview trap: "What is the default value of `logic`?" X. "Of `wire`?" Z. "Of `int`?" 0. "Of `integer`?" X. "Of a class handle?" `null`. "Of an enum?" The default of its base type, which may not be a valid member. Interviewers ask these as rapid-fire questions; know the table by heart.

Common mistake: Using a class handle, mailbox or virtual interface that is still at its default `null`. The result is a run-time "null object access" (null pointer) error, the most common run-time error in class-based testbenches.

Verification application: The first check in most testbenches is that DUT outputs leave X after reset. The four-state default is what makes that check possible: before reset, flip-flops are X; after reset, any output still at X points to a register that was not reset.

Interview questions

Q1. What is the default value of two-state and four-state variables? 0 and X respectively.

Q2. What is the default value of a `wire`, a `string`, a `real`, a class handle? Z, the empty string, 0.0, and `null`.

Q3. Why do four-state variables default to X and not 0? Because X models reality: a storage element that has not been initialised has an unknown value. Starting at 0 would hide missing-reset bugs.

Q4. What is the default size of a dynamic array? Zero. Memory is allocated with `new[n]`.

4.3 Four-State Logic Values: 0, 1, X, Z

Section 2.1 introduced the four values. This section explains exactly how X and Z behave in operators and statements, because that behaviour causes real bugs and many interview questions.

Meaning of each value

Value	In simulation	In real hardware	In synthesis
0	Logic low	Low voltage	Logic 0
1	Logic high	High voltage	Logic 1
X	Unknown: could be 0 or 1	Does not exist; the node has some voltage	Assigned in RTL (`y = 'x`) it means "don't care", letting the tool optimise
Z	Not driven	A floating node	Infers a tri-state buffer (`assign y = en ? d : 'z;`)

Writing X and Z in literals

```
4'b10xz        // individual x and z digits
8'hxF          // one hex x digit = four x bits : xxxx_1111
8'bz           // if the leftmost digit is x or z, it is used to pad : zzzz_zzzz
8'b1           // otherwise padding is with zeros : 0000_0001
'x  'z  '0  '1 // fill literals: every bit of the target, whatever its width
4'b1?0?        // ? is another way to write z in a literal (used in casez and ==? patterns)
```

X and Z are case-insensitive in literals: `4'bXXzz` is the same as `4'bxxZZ`.

Truth tables

For bitwise operators, Z on an input is treated exactly like X.

`&` (AND)	0	1	X	Z
0	0	0	0	0
1	0	1	X	X
X	0	X	X	X
Z	0	X	X	X

`|` (OR)	0	1	X	Z
0	0	1	X	X
1	1	1	1	1
X	X	1	X	X
Z	X	1	X	X

`^` (XOR)	0	1	X	Z
0	0	1	X	X
1	1	0	X	X
X	X	X	X	X
Z	X	X	X	X

`~` (NOT)	0	1	X	Z
Result	1	0	X	X

The way to remember them: a **controlling value** decides the output regardless of the other input. 0 controls AND (0 AND anything is 0). 1 controls OR (1 OR anything is 1). XOR has no controlling value, so any X gives X.

How X behaves in each kind of construct

Construct	Behaviour with X or Z	Example
Bitwise `&`, `|`, `^`, `~`	Bit by bit, using the tables above	`4'b1x00 & 4'b0011` = `0000`

Construct	Behaviour with X or Z	Example
Arithmetic `+ - * / % **`	**Any** X or Z bit in any operand makes the **whole** result X	`4'b1x00 + 1` = `xxxx`
Relational `< <= > >=`	Any X or Z bit makes the result `1'bx`	`4'b1x00 > 0` = `x`
Logical equality `==`, `!=`	`1'bx` if the outcome depends on an X or Z bit	See Section 4.5
Case equality `===`, `!==`	X and Z are compared literally; result is always 0 or 1	See Section 4.5
Logical `&&`, `||`, `!`	Operand is true if it has at least one 1 bit, false if all bits are 0, otherwise unknown; then the AND/OR tables apply	`1'b0 && 1'bx` = 0
Reduction `&`, `|`, `^`	The tables above applied across the bits	`&4'b1x0x` = 0, `|4'b1x0x` = 1
Shift by an X amount	Result is all X	
`if (cond)`	If `cond` is X or Z it is treated as **false**: the `else` branch runs	X-optimism
`cond ? a : b`	If `cond` is X: bits where `a` and `b` agree keep that value, the other bits become X	X-pessimism is partly modelled
`case`	Uses `===` matching: an X in the selector matches only an item with X in the same position; usually falls to `default`	
`while`, `for`, `wait` conditions	X is treated as false	
Edge `@(posedge clk)`	Transitions 0→1, 0→X, 0→Z, X→1, Z→1 all count as a posedge	
Array index	Read returns X (or 0 for two-state elements); write is ignored	

X-optimism and X-pessimism

- **X-optimism:** the simulation shows a known value where real hardware could have either value. The `if` statement is the main cause: with an unknown condition the simulator simply executes the `else` branch as though the condition were 0. A bug can be hidden because the simulation looks cleaner than reality.
- **X-pessimism:** the simulation shows X where real hardware would have a known value. Example: `y = (sel & a) | (~sel & a)` in gates. With `sel` = X and `a` = 1, the real output is 1 whichever value `sel` has, but the simulator computes `(X & 1) | (X & 1)` = X.

RTL simulation tends to be X-optimistic; gate-level simulation tends to be X-pessimistic. This difference is one reason gate-level simulations are still run.

Complete example

`tb_four_state.sv`

```
module tb_four_state;

  logic       sel;
  logic [3:0] a, b, y;

  initial begin
    a   = 4'b1100;
    b   = 4'b1010;
    sel = 1'bx;

    if (sel) y = a;
    else     y = b;
    $display("1. if-else with X select : y = %b", y);

    y = sel ? a : b;
    $display("2. ?: with X select      : y = %b", y);

    case (sel)
      1'b0   : $display("3. case: matched 0");
      1'b1   : $display("3. case: matched 1");
      default: $display("3. case: X matched neither 0 nor 1, default taken");
    endcase

    $display("4. 0&x=%b  1&x=%b  0|x=%b  1|x=%b  1^x=%b  ~x=%b  1&z=%b",
             1'b0 & 1'bx, 1'b1 & 1'bx, 1'b0 | 1'bx, 1'b1 | 1'bx, 1'b1 ^ 1'bx, ~1'bx,
             1'b1 & 1'bz);

    y = 4'b1x00;
    $display("5. 1x00 + 1 = %b", y + 4'b0001);
    $display("6. 1x00 > 0 = %b", y > 4'b0000);
    $display("7. &1x00 = %b   |1x00 = %b   ^1x00 = %b", &y, |y, ^y);
    $display("8. $isunknown(1x00) = %b   $countones(1x00) = %0d", $isunknown(y), $countones(y));
  end

endmodule
```

EXPECTED OUTPUT

```
1. if-else with X select : y = 1010
2. ?: with X select      : y = 1xx0
3. case: X matched neither 0 nor 1, default taken
4. 0&x=0  1&x=x  0|x=x  1|x=1  1^x=x  ~x=x  1&z=x
5. 1x00 + 1 = xxxx
6. 1x00 > 0 = x
7. &1x00 = 0   |1x00 = 1   ^1x00 = x
8. $isunknown(1x00) = 1   $countones(1x00) = 1
```

Line by line

Line	Explanation
1	`if (sel)` with `sel` = X is treated as false. The `else` branch executes and `y` gets `b`, a fully known value. The unknown select has disappeared. This is X-optimism.
2	The conditional operator merges both inputs. Bits 3 and 0 are the same in `a` and `b` (1 and 0), so they are known. Bits 2 and 1 differ, so they are X. This is a more accurate model of a multiplexer.
3	`case` compares with `===`. X is not identical to 0 or 1, so the `default` item runs. Without a `default`, nothing would execute.
4	Direct check of the truth tables. Note `1 & z` = X: Z at a gate input is unknown.

Line	Explanation
5	One X bit in an arithmetic operand makes every result bit X.
6	A relational operator with an X operand returns X.
7	Reduction AND sees a 0, so the answer is 0. Reduction OR sees a 1, so the answer is 1. Reduction XOR has no controlling value: X.
8	`$isunknown` detects X/Z. `$countones` counts only the bits that are 1.

Interview trap: "If `sel` is X, what does `if (sel) y = a; else y = b;` assign?" It assigns `b`. Many candidates answer "X". An `if` never produces X by itself; it just takes the `else` branch.

Common mistake: Believing `case` treats X as a wildcard. Plain `case` requires an exact four-state match. Only `casex` treats X and Z as don't-care, `casez` treats Z (and `?`) as don't-care, and `case ... inside` treats X/Z in the case items as don't-care.

Verification application: Testbenches include "X checks": an assertion or monitor check that control signals are never X or Z after reset, typically `assert property (@(posedge clk) disable iff (!rst_n) !$isunknown(valid));`. Because `if` and `==` are optimistic about X, an explicit `$isunknown` check is the reliable way to catch X on the DUT interface.

Interview questions

Q1. What is the difference between X and Z? X means the value is unknown: driven, but the simulator cannot tell whether it is 0 or 1 (uninitialised, conflicting drivers). Z means high impedance: not driven at all. Z read by logic behaves as X.

Q2. What is `1'b0 & 1'bx`? `1'b1 | 1'bx`? `1'b1 & 1'bx`? 0, 1 and X. A controlling value (0 for AND, 1 for OR) gives a known result despite the X.

Q3. What happens to `if (a)` when `a` is X? The condition is treated as false, so the `else` branch (if any) executes.

Q4. What is X-propagation / X-optimism? X-optimism is when RTL simulation turns an unknown into a known value (for example through `if` or `case`), hiding a potential bug that real hardware or gate-level simulation would expose. X-pessimism is the opposite: simulation reports X where hardware would have a definite value.

Q5. What is the result of `4'b101x + 4'b0001`? `4'bxxxx`. Arithmetic with any unknown bit gives an entirely unknown result.

Q6. Is X synthesizable? Assigning X in RTL is accepted by synthesis and is treated as a don't-care for optimisation. Comparing against X (`if (a === 1'bx)`) is not synthesizable, because hardware has no X to compare with.

4.4 Logical, Bitwise, Reduction and Shift Operators

All operators at a glance

Group	Operators	Operands	Result
Arithmetic	`+ - * / % **`	Two (also unary `+ -`)	A number
Increment / decrement	`++ --`	One variable	Modifies the variable
Logical	`&& || ! -> <->`	Treated as true/ false	1 bit: 0, 1 or X
Bitwise	`& | ^ ~ ~^ ^~`	Vectors, bit by bit	A vector as wide as the wider operand
Reduction	`& ~& | ~| ^ ~^ ^~` (unary)	One vector	1 bit
Shift	`<< >> <<< >>>`	Value, shift amount	A vector as wide as the left operand
Relational	`< <= > >=`	Two	1 bit
Equality	`== != === !== ==? !=?`	Two	1 bit
Conditional	`? :`	Three	One of the two values
Concatenation / replication	`{a, b}`, `{n{a}}`	Any number	A vector of the total width
Assignment	`= += -= *= /= %= &= |= ^= <<= >>= <<<= >>>=`		
Set membership	`inside`	Value, set	1 bit
Streaming	`{>>{}}`, `{<<{}}`		Packs / unpacks / reorders bits

Logical operators: && || !

Definition. Logical operators treat each operand as one truth value and return **one bit**.

- An operand is **true** if it is non-zero (at least one bit is 1).
- An operand is **false** if it is zero (all bits are 0).
- An operand is **unknown** if it has no 1 bit but has X or Z bits.

Operator	Meaning	Result
`a && b`	Logical AND	1 if both are true
`a || b`	Logical OR	1 if at least one is true
`!a`	Logical NOT	1 if `a` is false
`a -> b`	Logical implication	Same as `(!a) || b`
`a <-> b`	Logical equivalence	1 if both true or both false

Short-circuit evaluation. `&&`, `||`, `->` and `?:` evaluate their operands left to right and stop as soon as the result is known. In `a && b`, if `a` is false, `b` is **not evaluated**. This matters when `b` has a side effect or would be illegal to evaluate:

```
if (txn != null && txn.addr == 8'h10) ...      // txn.addr is not read when txn is null
if (idx < q.size() && q[idx] == 0) ...          // q[idx] is not read when idx is out of range
```

Bitwise operators: & | ^ ~ ~^

Definition. Bitwise operators work on each bit position independently and return a **vector**. If the operands have different widths, the narrower one is extended on the left first.

Operator	Meaning
`a & b`	AND of each bit pair
`a | b`	OR of each bit pair
`a ^ b`	XOR of each bit pair
`a ~^ b` or `a ^~ b`	XNOR of each bit pair
`~a`	Invert every bit

Logical versus bitwise

Property	Logical (`&&`, `||`, `!`)	Bitwise (`&`, `|`, `~`)
Operands are treated as	A single true/false value	A set of independent bits
Result width	1 bit	Width of the operands
`4'b1010 && 4'b0101` vs `4'b1010 & 4'b0101`	1 (both non-zero)	`0000`
`!4'b1010` vs `~4'b1010`	0	`0101`
Short-circuit	Yes	No, both operands are always evaluated
Typical use	Conditions in `if`, `while`, assertions	Masking, setting, clearing and toggling bits
For 1-bit operands	The two give the same value (but differ in short-circuit behaviour)	

Interview trap: `if (!a)` and `if (~a)` are the same only when `a` is one bit wide. For `a = 4'b1010`, `!a` is 0 (false), but `~a` is `0101`, which is non-zero and therefore true. Use `!` in conditions.

Interview trap: `bit flag = 1; int n = ~flag;` gives -2, not 0. The right-hand side is evaluated at the width of the destination (32 bits): `flag` is first extended to `0000...0001` and then inverted to `1111...1110`. With `!flag` the result is 0. Use `!` for a truth value and `~` only when you mean to invert bits.

Reduction operators

Definition. A reduction operator is a unary operator placed before a single vector. It applies the operation across all the bits of that vector and returns **one bit**.

Operator	Name	Returns 1 when	`a = 4'b1011`
`&a`	Reduction AND	Every bit is 1	0
`~&a`	Reduction NAND	Not every bit is 1	1
`|a`	Reduction OR	At least one bit is 1	1
`~|a`	Reduction NOR	Every bit is 0	0
`^a`	Reduction XOR	The number of 1 bits is odd (parity)	1
`~^a`	Reduction XNOR	The number of 1 bits is even	0

The same symbols `&`, `|`, `^` are bitwise when they have two operands and reduction when they have one.

Practical use	Expression
Even-parity bit	`parity = ^data;`
All ones?	`full = &count;`
Any bit set? (an interrupt pending)	`irq = |irq_status;`
All zero?	`empty = ~|count;`
Contains X or Z?	`(^data) === 1'bx`

Shift operators

Operator	Name	Vacated bits are filled with
`a << n`	Logical left shift	0
`a >> n`	Logical right shift	0
`a <<< n`	Arithmetic left shift	0 (identical to `<<`)
`a >>> n`	Arithmetic right shift	The sign bit if `a` is **signed**; 0 if `a` is unsigned

```
a = 8'b1001_0110  (signed: -106, unsigned: 150)

a << 2          ->  0101_1000     two zeros enter on the right; the top two bits are lost
a >> 2          ->  0010_0101     two zeros enter on the left
a >>> 2  signed ->  1110_0101     two copies of the sign bit enter on the left  (-27)
a >>> 2 unsigned->  0010_0101     same as >>
```

- The result has the width of the **left** operand. Bits shifted out are lost.
- The shift amount is always treated as unsigned.
- Shifting left by n multiplies by 2^n; shifting right by n divides by 2^n. For a signed value, `>>>` keeps the sign, but note that it rounds toward minus infinity (`-7 >>> 1` is -4) whereas `/` rounds toward zero (`-7 / 2` is -3).
- Shifts by a constant are free in hardware (just wiring). Shifts by a variable synthesise to a barrel shifter.

Arithmetic, relational and other operators in brief

Operator	Notes
`/`	Integer division truncates toward zero: `7 / 2` = 3, `-7 / 2` = -3
`%`	Remainder takes the sign of the first operand: `-7 % 2` = -1, `7 % -2` = 1. Not allowed on reals
`**`	Power: `2 ** 10` = 1024
`/ 0`, `% 0`	Result is X
`a ? b : c`	Selects `b` if `a` is true, `c` if false; merges if `a` is X
`{a, b}`	Concatenation. Every operand must have a known width, so unsized literals such as `1` are not allowed inside
`{n{a}}`	Replication: `{4{2'b10}}` = `8'b10101010`
`i++`, `++i`, `i += 2`	These are **blocking** assignments. Use them for testbench variables and loop counters, not for flip-flops in `always_ff`
`x inside {1, 2, [5:9]}`	True if `x` equals any listed value or lies in a listed range. Uses `==?` matching, so X/Z in the set are wildcards

Operator precedence

Highest precedence first. Operators on the same row have equal precedence and associate left to right, except `?:`, `->`, `<->` and the assignment operators, which associate right to left.

Level	Operators
1 (highest)	`()` `[]` `::` `.`
2	Unary `+` `-` `!` `~` `&` `~&` `|` `~|` `^` `~^` `++` `--`
3	`**`
4	`*` `/` `%`
5	Binary `+` `-`
6	`<<` `>>` `<<<` `>>>`
7	`<` `<=` `>` `>=` `inside` `dist`
8	`==` `!=` `===` `!==` `==?` `!=?`
9	Binary `&`
10	Binary `^` `~^` `^~`
11	Binary `|`
12	`&&`
13	`||`
14	`?:`
15	`->` `<->`
16	`=` `+=` `-=` and the other assignment operators
17 (lowest)	`{}` `{{}}`

> **Interview trap:** `if (a & b == c)` is parsed as `if (a & (b == c))`, because `==` has higher precedence than `&`. Write `if ((a & b) == c)`. When in doubt, add parentheses; nobody is impressed by code that relies on precedence.

Expression width: the rule behind many bugs

Before an expression is evaluated, SystemVerilog decides how many bits to use.

1. For "context-determined" operands (operands of `+ - * / % & | ^`, the left operand of a shift, both result operands of `?:`), the width is the **largest** width among all those operands **and the left-hand side** of the assignment.
2. Every such operand is extended to that width **before** the operation.
3. "Self-determined" operands use only their own width: the shift amount, operands inside a concatenation, operands of a comparison (which are sized against each other only), and arguments to `$display`.

```
   logic [7:0] p = 200, q = 100;
   logic [8:0] sum9;
   logic [7:0] avg;

   sum9 = p + q;            width = max(8, 8, 9) = 9   ->  300        carry kept
   avg  = (p + q) >> 1;     width = max(8, 8, 8) = 8   ->  300 mod 256 = 44, then 44 >> 1 = 22   WRONG
   avg  = ({1'b0, p} + q) >> 1;   width = 9            ->  300 >> 1 = 150
correct
```

Complete example

tb_operators.sv

```
module tb_operators;

  logic [3:0] a, b;
  logic [7:0] p, q, avg;
  bit         flag;
  int         n;

  initial begin
    a = 4'b1010;
    b = 4'b0101;
    $display("1.  logical : a && b = %b   a || b = %b   !a = %b", a && b, a || b, !a);
    $display("2.  bitwise : a & b = %b   a | b = %b   a ^ b = %b   ~a = %b",
             a & b, a | b, a ^ b, ~a);

    a = 4'b1011;
    $display("3.  reduction of 1011 : &a=%b  |a=%b  ^a=%b  ~&a=%b  ~|a=%b  ~^a=%b",
             &a, |a, ^a, ~&a, ~|a, ~^a);
    $display("4.  shift of 1011     : a << 1 = %b   a >> 1 = %b", a << 1, a >> 1);

    $display("5.  7 / 2 = %0d   -7 / 2 = %0d   -7 %% 2 = %0d   2 ** 10 = %0d",
             7 / 2, -7 / 2, -7 % 2, 2 ** 10);
    $display("6.  a / 0 = %b", a / 4'd0);

    flag = 1;
    n = ~flag;   $display("7.  int n = ~flag -> %0d", n);
    n = !flag;   $display("8.  int n = !flag -> %0d", n);

    p = 8'd200;
    q = 8'd100;
    avg = (p + q) >> 1;             $display("9.  (p + q) >> 1          = %0d", avg);
    avg = ({1'b0, p} + q) >> 1;     $display("10. ({1'b0, p} + q) >> 1  = %0d", avg);

    $display("11. max = %0d", (p > q) ? p : q);
    $display("12. {a, b} = %b   {2{a}} = %b", {a, b}, {2{a}});

    if (a inside {4'd3, [4'd9:4'd12]}) $display("13. a (=%0d) is inside {3, [9:12]}", a);

    $display("14. a & b == b   -> %b", a & b == b);
    $display("15. (a & b) == b -> %b", (a & b) == b);
  end

endmodule
```

EXPECTED OUTPUT

```
1.  logical : a && b = 1   a || b = 1   !a = 0
2.  bitwise : a & b = 0000   a | b = 1111   a ^ b = 1111   ~a = 0101
3.  reduction of 1011 : &a=0  |a=1  ^a=1  ~&a=1  ~|a=0  ~^a=0
4.  shift of 1011     : a << 1 = 0110   a >> 1 = 0101
5.  7 / 2 = 3   -7 / 2 = -3   -7 % 2 = -1   2 ** 10 = 1024
6.  a / 0 = xxxx
7.  int n = ~flag -> -2
8.  int n = !flag -> 0
9.  (p + q) >> 1          = 22
10. ({1'b0, p} + q) >> 1  = 150
11. max = 200
12. {a, b} = 10110101   {2{a}} = 10111011
13. a (=11) is inside {3, [9:12]}
14. a & b == b   -> 0001
15. (a & b) == b -> 0
```

Line by line

Line	Explanation
1	Both `a` and `b` are non-zero, so each is "true". `a && b` = 1, `a || b` = 1, `!a` = 0. One-bit results.
2	Bit-by-bit: `1010 & 0101` = `0000`, OR = `1111`, XOR = `1111`, `~1010` = `0101`. Compare with line 1: the logical AND is 1 while the bitwise AND is 0.
3	`1011`: not all ones → `&` = 0; some ones → `|` = 1; three ones (odd) → `^` = 1. The last three are the inverses.
4	Inside `$display` the expression is 4 bits wide (self-determined). `1011 << 1` loses the top bit: `0110`. `1011 >> 1` = `0101`.
5	Integer division truncates toward zero; the remainder has the sign of the first operand.
6	Division by zero gives X in every bit.
7	`flag` is extended to 32 bits, then inverted: `FFFF_FFFE` = -2.
8	`!flag` is the 1-bit value 0, extended to 32 bits: 0.
9	Width is 8 bits. 200 + 100 = 300 overflows to 44; 44 >> 1 = 22. The carry was lost **before** the shift.
10	The concatenation `{1'b0, p}` is 9 bits wide, so the addition is done in 9 bits: 300 >> 1 = 150.
11	Conditional operator used as "maximum".
12	Concatenation joins `1011` and `0101`. Replication repeats `1011` twice.
13	`a` is 11, which lies in the range 9 to 12.
14	Parsed as `a & (b == b)`. `b == b` is 1; `1011 & 0001` = `0001`.
15	`(1011 & 0101)` = `0001`, which is not equal to `0101`: 0. Lines 14 and 15 show why parentheses are needed.

Common mistake: Writing `a = b << 1 + 1;` and expecting `(b << 1) + 1`. Addition has higher precedence than shift, so this is `b << 2`.

Verification application: Reference models in a scoreboard must reproduce the DUT's widths exactly. For the ALU, the model computes `exp = {1'b0, a} + {1'b0, b};` into a 9-bit variable so that the carry is modelled. A model that computes in 8 bits will mismatch on every overflow, and a model that computes in 32-bit `int` will mismatch on every subtraction that borrows unless the result is truncated to 9 bits.

Interview questions

Q1. What is the difference between `&&` and `&`? `&&` is logical AND: each operand is reduced to true or false and the result is one bit, with short-circuit evaluation. `&` with two operands is bitwise AND: each pair of bits is ANDed and the result is a vector.

Q2. What is the difference between `!` and `~`? `!` is logical negation and gives a 1-bit result: 1 if the operand is zero. `~` inverts every bit of the operand and gives a result of the same width. They are equivalent only for 1-bit operands.

Q3. What is a reduction operator? Give a use. A unary operator that combines all bits of one vector into a single bit, for example `^data` to compute parity, `&vec` to test for all ones, or `|vec` to test for any bit set.

Q4. With `a = 4'b0110` and `b = 4'b1001`, what are `a & b`, `a && b`, `a | b`, `a || b`, `&a`, `|a`, `^a`? `0000`, 1, `1111`, 1, 0, 1, 0 (two ones, even parity).

Q5. What is the difference between `>>` and `>>>`? And between `<<` and `<<<`? `>>` always fills with zeros. `>>>` fills with the sign bit when the left operand is signed. `<<` and `<<<` are identical.

Q6. What is short-circuit evaluation? For `&&`, `||`, `->` and `?:`, evaluation stops as soon as the result is determined, so the remaining operand is not evaluated. It is commonly used to guard an access: `if (h != null && h.valid)`.

Q7. `logic [7:0] a = 8'hFF, b = 8'h01; logic [7:0] s; s = (a + b) >> 1;` What is `s`? 0. The expression is evaluated in 8 bits, so `FF + 01` wraps to `00`, and `00 >> 1` is `00`. To get `80`, widen an operand: `s = ({1'b0, a} + b) >> 1;`.

Q8. What is the result of `-7 % 3` and `7 % -3`? -1 and 1. The result of `%` takes the sign of the first operand.

Q9. Which has higher precedence, `==` or `&`? `==`. So `a & b == c` means `a & (b == c)`.

Q10. How would you swap the upper and lower nibbles of a byte in one statement? `data = {data[3:0], data[7:4]};`

4.5 Equality Operators: ==, ===, !=, !==

In plain English

There are two ways to ask "are these two values the same?".

- **Logical equality** `==` answers as hardware would: if an operand contains an unknown bit that matters, the honest answer is "I do not know" (X).
- **Case equality** `===` answers as a text comparison would: it compares the four-state symbols literally, so X equals X and Z equals Z, and the answer is always a definite yes or no.

Technical definition

Operator	Name	How X and Z in the operands are handled	Possible results
`a == b`	Logical equality	If the result depends on an X or Z bit, the result is X	0, 1, X
`a != b`	Logical inequality	Same	0, 1, X
`a === b`	Case equality (four-state equality)	X and Z are compared as ordinary values and must match exactly	0, 1
`a !== b`	Case inequality	Same	0, 1

How == works, step by step

1. Operands of different width are extended to the same width.
2. If any pair of **known** bits (0/1) differs, the values are definitely different: result **0**.
3. Otherwise, if any bit of either operand is X or Z, the result is ambiguous: result **X**.
4. Otherwise all bits are known and equal: result **1**.

`===` simply compares each bit position for an identical symbol (0, 1, X or Z) and returns 1 only if all positions match.

Single-bit truth tables:

==	0	1	X	Z
0	1	0	X	X
1	0	1	X	X
X	X	X	X	X
Z	X	X	X	X

===	0	1	X	Z
0	1	0	0	0
1	0	1	0	0
X	0	0	1	0
Z	0	0	0	1

Complete example

tb_equality.sv

```
module tb_equality;

  logic [3:0] a, b, c;
  logic [7:0] actual, expected;

  initial begin
    a = 4'b10x1;
    b = 4'b10x1;
    c = 4'b00x1;

    $display("1. a == b : %b    a === b : %b", a == b, a === b);
    $display("2. a != b : %b    a !== b : %b", a != b, a !== b);
    $display("3. a == c : %b    a === c : %b", a == c, a === c);

    if (a == b) $display("4. if (a == b) : taken");
    else        $display("4. if (a == b) : NOT taken, X is treated as false");

    if (a != b) $display("5. if (a != b) : taken");
    else        $display("5. if (a != b) : NOT taken either");

    // The scoreboard trap: the DUT output is X
    expected = 8'h3C;
    actual   = 8'hxx;

    if (actual != expected)  $display("6. using != : MISMATCH reported");
    else                     $display("6. using != : no mismatch reported  <-- bug hidden");

    if (actual !== expected) $display("7. using !== : MISMATCH reported    <-- correct");
    else                     $display("7. using !== : no mismatch reported");
  end

endmodule
```

EXPECTED OUTPUT

```
1. a == b : x    a === b : 1
2. a != b : x    a !== b : 0
3. a == c : 0    a === c : 0
4. if (a == b) : NOT taken, X is treated as false
5. if (a != b) : NOT taken either
6. using != : no mismatch reported  <-- bug hidden
7. using !== : MISMATCH reported    <-- correct
```

Line by line

Line	Explanation
1	`a` and `b` are both `10x1`. With `==`, the X bit makes the answer unknown: X. With `===`, X matches X: 1.
2	The same for inequality: `!=` gives X, `!==` gives a definite 0.
3	`a` and `c` differ in bit 3 (1 versus 0), both known. They cannot be equal whatever the X is, so `==` gives a definite 0.
4, 5	Both `a == b` and `a != b` evaluate to X, and `if (X)` is false. **Neither** branch condition is true. `a == b` and `a != b` are not opposites when X is involved.
6	The DUT output is all X. `actual != expected` is X, the `if` is not taken, and the testbench reports nothing. The test passes with a completely broken output.
7	`!==` returns 1 because X is not identical to the expected bits. The error is reported.

Which operator to use where

Situation	Operator	Reason
RTL (synthesizable code)	`==`, `!=`	`===` has no hardware meaning; synthesis tools reject it or treat it as `==`
Scoreboard / checker comparing DUT output with expected	`!==` / `===`	X or Z on the DUT output must count as a mismatch
Checking explicitly for X or Z	`===`, or `$isunknown()`	`sig == 1'bx` is always X and therefore never true
Comparing two-state values (`bit`, `int`)	`==`	No X possible, so both operators behave the same
Comparing class handles	`==`, `!=`	Compares whether both handles refer to the same object, not the object contents (Module 4)
Comparing strings, unpacked arrays, unpacked structs	`==`, `!=`	Compared element by element
Comparing reals	`==` with caution	`===` is not allowed on reals; prefer a tolerance comparison

Relationship with case statements

Statement	Comparison used
`case`	`===`: exact four-state match
`casez`	Z and `?` bits (in the selector or the item) are don't-care
`casex`	X, Z and `?` bits (in the selector or the item) are don't-care

Statement	Comparison used
`case (...) inside`	`==?` : X, Z and `?` bits **in the item only** are don't-care

Interview trap: "What does `if (sig == 1'bx)` do?" It is never true. Comparing anything with X using `==` gives X, which an `if` treats as false. To test for X use `if (sig === 1'bx)` or `if ($isunknown(sig))`.

Interview trap: "Is `!(a == b)` the same as `a != b`?" In value, yes: both are X when the comparison is ambiguous. But candidates often assume that if `a == b` is false then `a != b` must be true. With X, both can be "not true".

Verification application: Every scoreboard comparison of DUT data should be written as `if (actual !== expected) $error(...)`. This single habit prevents the most damaging kind of testbench bug: a test that passes while the DUT outputs X.

Interview questions

Q1. What is the difference between `==` and `===`? `==` is logical equality: if either operand has X or Z bits that affect the outcome, the result is X. `===` is case equality: it compares all four states literally, so X matches X and Z matches Z, and the result is always 0 or 1.

Q2. `a = 4'b1x01; b = 4'b1x01;` **What are `a == b` and `a === b`?** X and 1.

Q3. `a = 4'b1x01; b = 4'b0x01;` **What is `a == b`?** 0. Bit 3 is known and different, so the values cannot be equal regardless of the X.

Q4. Which equality operator is synthesizable? `==` and `!=`. `===` and `!==` are for simulation, because real hardware cannot test for X or Z.

Q5. Which would you use in a scoreboard and why? `===` / `!==`, so that an X or Z on the DUT output is detected as a mismatch. With `!=` the comparison would be X, the `if` would be skipped, and the failure would go unreported.

Q6. How do you check whether a signal is X? `$isunknown(sig)`, or `sig === 'x` for an exact all-X match. Not `sig == 'x`.

Q7. What does `==` return when comparing two-state operands? Always 0 or 1. X is only possible when an operand is four-state.

4.6 Wildcard Equality: ==? and !=?

In plain English

Sometimes only part of a value matters: "is the address in the range `A0xx`?", "is the top bit of the opcode 1?". The wildcard equality operator compares two values while **ignoring the bit positions that are marked as don't-care in the right-hand operand**.

Technical definition

Operator	Name	Behaviour
`a ==? b`	Wildcard equality	Bit positions where `b` (the **right** operand) is X or Z are not compared. The remaining positions are compared like `==`
`a !=? b`	Wildcard inequality	The logical inverse of `==?`

Three points define the operator completely:

1. **Only the right-hand operand supplies wildcards.** X or Z (also written `?`) in `b` means "don't care about this bit".
2. **X or Z in the left-hand operand is not a wildcard.** If the left operand has X/Z in a position that is being compared, the result is X, as with `==`.
3. The result can be 0, 1 or X.

So the operator is **asymmetric**: `a ==? b` and `b ==? a` can give different results.

How it works, step by step

```
a ==? b

1. Build a mask from b:   mask bit = 0 where b is X or Z,  1 where b is 0 or 1
2. Compare only the positions where mask = 1, using the rules of ==

Example:   a = 1010,  b = 1xx0

     a       :  1  0  1  0
     b       :  1  x  x  0
     mask    :  1  0  0  1          <- only bit 3 and bit 0 are compared
     compare:   =  -  -  =          -> result 1
```

Complete example

tb_wildcard.sv

```
module tb_wildcard;

  logic [3:0]  a;
  logic [15:0] addr;

  initial begin
    a = 4'b1010;
    $display("1. 1010 ==? 1xx0 : %b", a ==? 4'b1xx0);
    $display("2. 1010 ==? 0xxx : %b", a ==? 4'b0xxx);
    $display("3. 1010 ==? 1?1? : %b", a ==? 4'b1?1?);
    $display("4. 1010 !=? 1xx0 : %b", a !=? 4'b1xx0);

    a = 4'b1x10;
    $display("5. 1x10 ==? 1010 : %b", a ==? 4'b1010);
    $display("6. 1x10 ==? 1?10 : %b", a ==? 4'b1?10);
    $display("7. 1x10 ==? 0xxx : %b", a ==? 4'b0xxx);

    $display("8. 1010 ==? 1xx0 : %b   but   1xx0 ==? 1010 : %b",
             4'b1010 ==? 4'b1xx0, 4'b1xx0 ==? 4'b1010);

    addr = 16'hA07C;
    if (addr ==? 16'hA0??) $display("9. address %h is in the A0xx block", addr);

    a = 4'b1010;
    if (a inside {4'b0???, 4'b1??0}) $display("10. inside also uses wildcard matching");
  end

endmodule
```

EXPECTED OUTPUT

```
1. 1010 ==? 1xx0 : 1
2. 1010 ==? 0xxx : 0
3. 1010 ==? 1?1? : 1
4. 1010 !=? 1xx0 : 0
5. 1x10 ==? 1010 : x
6. 1x10 ==? 1?10 : 1
7. 1x10 ==? 0xxx : 0
8. 1010 ==? 1xx0 : 1   but   1xx0 ==? 1010 : x
9. address a07c is in the A0xx block
10. inside also uses wildcard matching
```

Line by line

Line	Explanation
1	Bits 2 and 1 of the right operand are X, so only bits 3 and 0 are compared: 1=1 and 0=0. Match.
2	Only bit 3 is compared: 1 versus 0. No match.
3	`?` in a literal means Z, which is also a wildcard. Bits 3 and 1 are compared: 1=1, 1=1. Match.
4	`!=?` is the inverse of line 1.
5	The right operand has no wildcards, so all bits are compared. The **left** operand has an X in bit 2, which is not a wildcard, so the result is X.
6	Now the right operand has a wildcard in bit 2, which masks the X on the left. The other bits match: 1.
7	Bit 3 is 1 versus 0, known and different: 0. The X on the left is in a masked position and does not matter.
8	Asymmetry: with the pattern on the right the answer is 1; with the pattern on the left it is X.

Line	Explanation
9	In a hex literal each `?` stands for four Z bits, so `16'hA0??` matches any address whose upper byte is `A0`.
10	The `inside` operator compares with `==?`, so X, Z and `?` in the set are wildcards. `1010` matches `1??0`.

Comparison of the three equality families

Using `a = 4'b10x1`:

Expression	`==`	`===`	`==?`
`a` op `4'b10x1`	X	1	1 (the X on the right masks bit 1)
`a` op `4'b1001`	X	0	X (the X on the left is compared)
`a` op `4'b00x1`	0	0	0
`4'b1011` op `4'b10x1`	X	0	1

Property	`==` / `!=`	`===` / `!==`	`==?` / `!=?`
X/Z in the right operand	Makes the result X	Must match exactly	Don't-care (wildcard)
X/Z in the left operand	Makes the result X	Must match exactly	Makes the result X (if in a compared position)
Possible results	0, 1, X	0, 1	0, 1, X
Symmetric	Yes	Yes	No
Synthesizable	Yes	No	Yes, when the right operand is a constant pattern
Typical use	RTL and general comparison	Testbench checks that must see X/Z	Pattern matching on part of a value

Where wildcard matching appears in verification

Construct	Wildcard behaviour
`==?`, `!=?`	X/Z/`?` in the right operand
`inside`	X/Z/`?` in the values of the set
`case (...) inside`	X/Z/`?` in the case items
`casez` / `casex`	Z (and X for `casex`) on **both** sides, which is why `casex` is considered dangerous in RTL
`wildcard bins` in a coverpoint	X/Z/`?` in the bin value: `wildcard bins odd = {4'b???1};` (Module 10)

Interview trap: "Does `==?` ignore X in both operands?" No. Only X and Z in the **right** operand are wildcards. An X in the left operand gives an X result. That one-sidedness is the difference between `==?` (and `case inside`) and `casex`.

Common mistake: Putting the pattern on the left: `if (4'b1??0 ==? a)`. The wildcards are then on the wrong side and are treated as real Z values; the result is X and the `if` never executes.

Interview questions

Q1. What does `==?` do? It compares two values but ignores the bit positions in which the right-hand operand is X or Z. It is used to match a value against a pattern with don't-care bits.

Q2. What is the difference between `==`, `===` and `==?`? `==` returns X if any relevant bit is X/Z. `===` compares X and Z literally and always returns 0 or 1. `==?` treats X/Z in the right operand as wildcards and otherwise behaves like `==`.

Q3. Is `a ==? b` the same as `b ==? a`? No. Only the right operand provides wildcards, so the operator is asymmetric.

Q4. `4'b1x01 ==? 4'b1001`: what is the result? X. The X is in the left operand, where it is not a wildcard, and the right operand masks nothing.

Q5. How is `==?` related to `inside` and to wildcard bins? `inside` uses `==?` to compare the value with each member of the set, and `case inside` does the same with its items, so X/Z/`?` in the set or items act as wildcards. Wildcard bins in functional coverage apply the same idea to bin definitions.

Q6. Is `==?` synthesizable? Yes, when the wildcard pattern is a constant; the tool simply omits the don't-care bits from the comparator.